
CS 61B

Fall 2024

Final

Thursday, December 19, 2024

Your Name: [Solutions](#). Last updated: Mon, Dec 15, 2025 (changelog: [Natalia MST got solutions –peyrin](#))

Your SID: _____ Location: _____

SID of person to your left: _____ Right: _____

This exam is worth **100 points**, and consists of 6 questions. Questions are ordered by topic, not by difficulty.

Question:	1	2	3	4	5	6	Total
Points:	8	15	21	18	16	22	100

- ☐ indicates only one circle should be filled in. ☐ indicates more than one box may be filled in. **Please fill in the shape completely.** If you change your response, **erase as completely as possible.**
- Anything you write that you ~~cross out~~ will not be graded.
- If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade according to the worst interpretation.
- Unless otherwise specified, all data structures and algorithms behave according to their implementation in lecture, with no additional optimizations.
- If an implementation detail (e.g. tiebreaking scheme, linked list topology) is relevant, it will be explicitly noted in the question.

Coding questions:

- You may write at most one statement per blank and you may not use more blanks than provided.
- Your answer will be reformatted according to the 61B style guidelines. For example, any if-statement requires at least three lines for the purposes of determining line count.
- You may not use ternary operators, lambdas, streams, or multiple assignment.
- Unless otherwise specified, you may assume that everything on the reference card has been imported, and that all code within a question belongs to the same class.

Write the statement below in the same handwriting you will use on the rest of the exam. Failure to do so may result in a voided exam.

I have neither given nor received help on this exam (or quiz), and have rejected any attempt to cheat. If these answers are not my own work, I may be deducted up to 9,876,543,210 points.

Signature: _____

1 Potpourri

(8 points)

- (a) (1.5 points) True/False: Iterators in a Java program use the
- `yield`
- keyword instead of
- `return`
- .

☐ True☒ FalseFalse. `yield` is a Python keyword, not a Java keyword.In Java, we create an iterator by creating an object that extends `Iterator`. This object has a `next` method, which returns the next item when you call the method.

- (b) (1.5 points) True/False: If a function receives as input a list of length
- N
- , its runtime must be at least
- $\Omega(N)$
- .

☐ True☒ False

False. Consider this function:

```
public void f1(int[] arr) {
    return arr[0];
}
```

If we pass in an array of length N , the runtime is $\Theta(1)$. $\Omega(N)$ is the family of all functions greater than or equal to N , e.g. it includes N^2 , 2^N , $N \log N$, etc. This family does not include 1. Thus, $\Theta(1)$, the runtime of this function, is not in $\Omega(N)$.

- (c) (2 points) What is the runtime of determining whether an unsorted array of
- N
- integers contains the integer 1? Select all true statements.

☒ Best-case $O(1)$ ☐ Best-case $\Omega(N)$ ☒ Worst-case $\Omega(\log(N))$ ☒ Worst-case $\Theta(N)$ ☐ None of the above

First, we can get big-Theta bounds on the best-case and worst-case runtime. Then, we can think about what big-O or big-Omega families that our bound belongs to.

Best Case:One best-case scenario is if the integer 1 is at the very start of the array, and we scan the array from front-to-back. (Other best-case scenarios exist too.) Then the runtime is $\Theta(1)$. $O(1)$ is the family of all functions less than or equal to 1, which is really just 1. This family does include $\Theta(1)$, so Option 1 is true. $\Omega(N)$ is the family of all functions greater than or equal to N . This family includes N^2 , N^3 , $N \log N$, and many others. However, it does not include 1. Therefore, Option 2 is false.**Worst Case:**

One worst-case scenario is if the integer 1 is at the very end of the array, and we scan the array from front-to-back. (Other worst-case scenarios exist too.) Then the runtime is $\Theta(N)$, because we scan the entire array. This tells us that Option 4 is true.

$\Omega(\log N)$ is the family of all functions greater than or equal to $\log N$. This family includes N^2 , $N \log N$, and indeed N as well. Because N lives in this family, we can say that the worst-case runtime is $\Omega(\log N)$, and Option 3 is true.

- (d) (1.5 points) True/False: A Left-Leaning Red-Black Tree is always exactly twice as tall as its corresponding B-Tree.

☐ True

☒ False

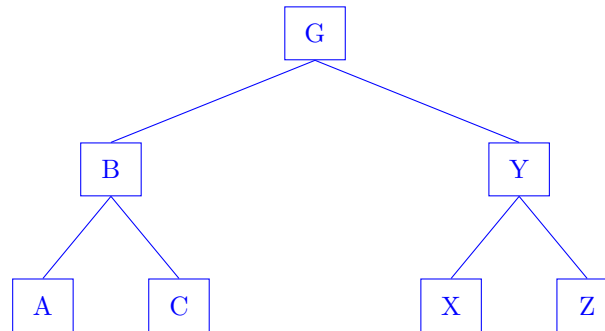
False.

Counter-Example 1:

As a counterexample, consider a 2-3-tree (limit 2 items per node) where every node is a 2-node (node with 1 item and 2 children).

The corresponding LLRB is exactly the same (no red links). The height of this LLRB is exactly the same, not twice as tall.

Below is an example of a B-tree where every node is a 2-node. The corresponding LLRB is exactly the same, with no red links.

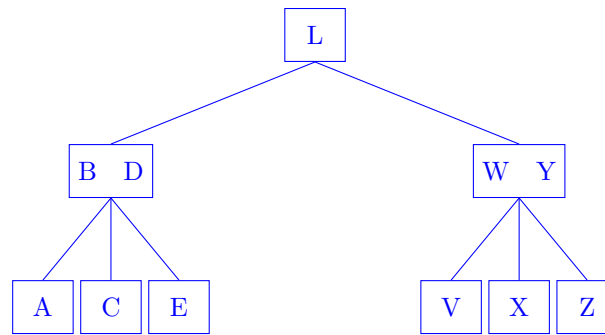


Counter-Example 2:

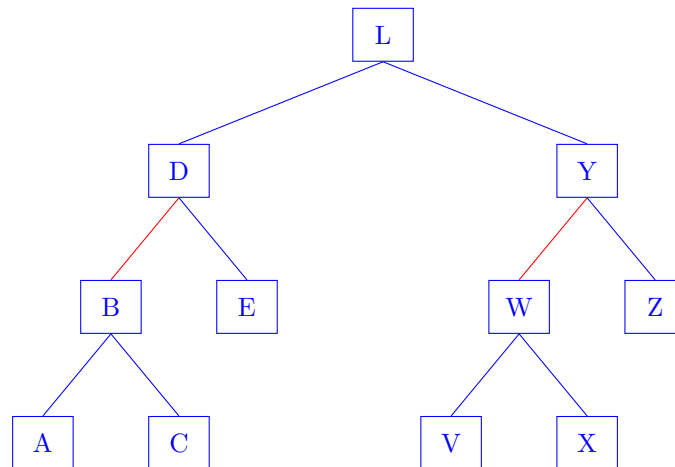
In general, the LLRB invariant says: An LLRB is **at most roughly** twice as tall as its corresponding B-Tree.

However, there are many cases where the LLRB is taller than the corresponding B-Tree, but not exactly twice as tall.

Below is an example of a 2-3-tree (limit 2 items per node) with height 2:



But if we convert this 2-3-tree to its corresponding LLRB (red links are colored red), the corresponding LLRB has height 3 (not 4):



Note: In 61B, we say that the depth of a node is the number of links between that node and the root, e.g. C has depth 3 in the LLRB above, and L has depth 0. We say that the height of a tree is the depth of its deepest node, which is why the LLRB above has height 3.

(e) (1.5 points) True/False: In a Java program, calling `null.hashCode()` will result in an error.

☒ True

☐ False

True. You cannot call methods on `null`, because it is not an object. `null` indicates the non-existence of an object.

Remember that when we say something like `Dog d = new Dog();`, the variable `d` is a reference type, which means that `d` holds the memory address of where the actual `Dog` is stored. We can then access variables like `d.weight`, which go to the memory address to find the actual `Dog`'s weight.

However, if we say `Dog d = null;`, the variable `d` is holding all zeroes, to indicate that there is no actual `Dog` in memory that `d` is referencing. Therefore, you cannot access variables like `d.weight`, since `d` is not pointing to any actual `Dog` object.

Similarly, a method call like `d.bark()` needs to reference the actual instance of `Dog` in memory, but if `d` is `null`, then there is no actual instance of `Dog` that we can reference. Therefore, Java will error if you try to call a method on `null`.

2 Do It Aditya's Way!

(15 points)

Aditya is trying to come up with a way to compare superclass-subclass relationships. He decides that a superclass should be “greater than” a subclass. For example, for the following classes Glorp, Foo, and Snapp:

```
public class Glorp {...}
public class Foo extends Glorp {...}
public class Snapp extends Glorp {...}
```

We would say that $\text{Glorp} > \text{Foo}$, and $\text{Glorp} > \text{Snapp}$.

Snapp and Foo are *incomparable*; $\text{Foo} \not> \text{Snapp}$, $\text{Snapp} \not> \text{Foo}$, and $\text{Foo} \neq \text{Snapp}$.

(a) (1.5 points) True/False: If class $A > B$, and class $B > C$, then class $A > C$.

☒ True

☐ False

One way to reason through this is to think about “**is-a**” inheritance relationships.

When we say that Dog extends Animal, we are saying that a dog **is-a** Animal.

Also, the question says that if Dog extends Animal, then $\text{Animal} > \text{Dog}$.

This allows us to reword the question as: If B **is-a** A, and C **is-a** B, then C **is-a** A.

Plugging in some intuitive names: If Dog **is-a** Animal, and ShowDog **is-a** Dog, then ShowDog **is-a** Animal. This statement is true.

A **full ordering** of a list of classes is any ordering of classes such that, if a class $A > B$, class A appears after class B.

(b) (1.5 points) True/False: If class A appears after class B in a full ordering, then necessarily $A > B$.

☐ True

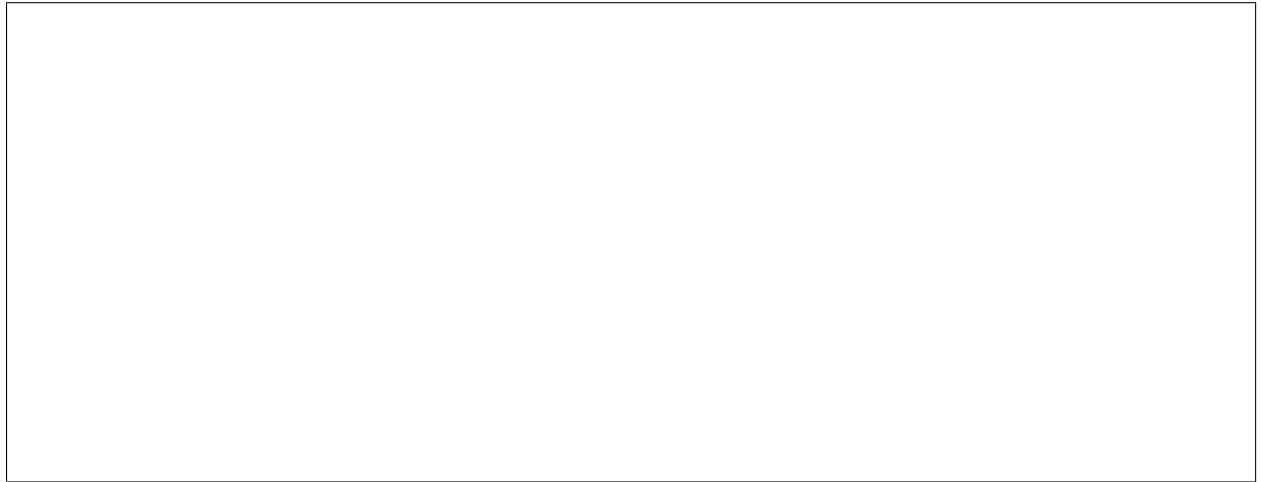
☒ False

False. For a counterexample, consider three classes: Animal, Dog, Potato. Suppose that Dog extends Animal (i.e. $\text{Animal} > \text{Dog}$) is the only inheritance relationship between these three classes.

Dog, Animal, Potato is a valid full ordering. We know that $\text{Animal} > \text{Dog}$, and Animal appears after Dog in the full ordering.

In this ordering, Potato appears after Dog. However, it is not true that $\text{Potato} > \text{Dog}$ (i.e. Dog does not extend Potato). Therefore, the statement is false.

(c) (6 points) Design an algorithm for finding a full ordering given a set of classes (where each class is represented as a `String`) and a list of superclass-subclass pairs within the set. This algorithm must run in $O(N + M)$ time, where N is the number of classes and M is the total number of superclass-subclass pairs. Your algorithm may return any full ordering of classes. Describe your algorithm in no more than three sentences; no code is needed.

**Algorithm:**

Create a directed acyclic graph (DAG), where each class is a node, and each superclass-subclass pair is represented as an arrow pointing from the subclass to the superclass.

Then, perform a topological sort on the DAG (or equivalently, find the reverse of the DFS post-order). The resulting order of vertices gives the desired sort.

Explanation:

Recall that topological sorting produces an ordering of nodes, such that all the directed edges point toward the right.

If $A > B$, we can draw an arrow $B \rightarrow A$.

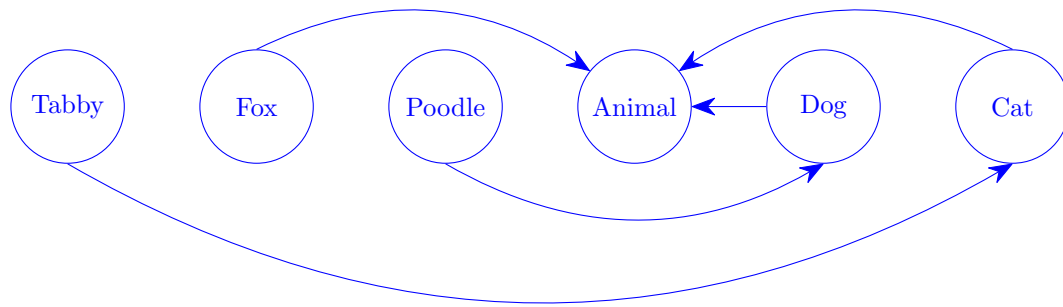
In the topological sort, this edge will point to the right, which means that A will appear after B in the topological sort.

Here's a concrete example. Suppose we have the classes Tabby, Fox, Poodle, Animal, Dog, Cat, and the following superclass-subclass relationships:

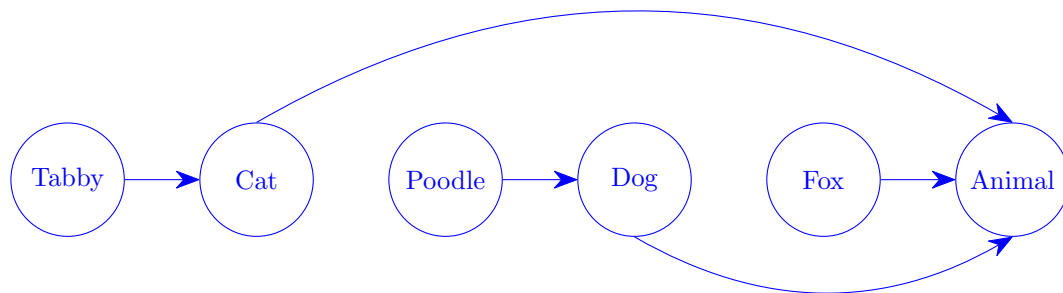
- $\text{Animal} > \text{Dog}$
- $\text{Animal} > \text{Cat}$
- $\text{Animal} > \text{Fox}$
- $\text{Dog} > \text{Poodle}$
- $\text{Cat} > \text{Tabby}$

By the transitive property described in subpart (a), you could also derive $\text{Animal} > \text{Poodle}$ and $\text{Animal} > \text{Tabby}$, but we'll omit those here for simplicity.

From these classes and these relationships, we'll build a DAG that looks like this:



This DAG has many valid topological sorts, but one example of a valid topological sort is:



We've arranged the nodes such that all the arrows point toward the right.

If you read the nodes from left-to-right, you get a full ordering of the classes, as desired. For example, Cat comes after Tabby, which is consistent with the fact that $\text{Cat} > \text{Tabby}$.

Runtime Proof:

This DAG has N vertices (one per class) and M edges (one per superclass-subclass pair), so topologically sorting this graph takes $\Theta(N + M)$ time (topological sort can be done with DFS, which takes $\Theta(V + E)$ time). Therefore, our solution meets the runtime requirement. You were not required to prove the runtime bound on the exam.

Alternate solutions:

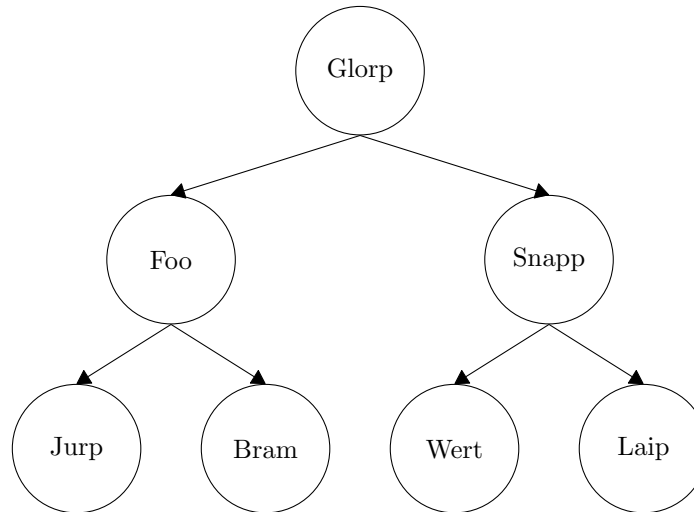
You can swap the direction of all arrows, and find the reverse topological sort. For example, create a DAG where the edges point from superclass to subclass. Then, find the reverse topological sort (or, equivalently, find the DFS post-order) of this DAG.

- (d) (1 point) Given any full ordering of every non-primitive class that exists in a particular Java program, which class will always be the final element in the ordering? Answer in no more than 2 words.

Object. Every class in Java extends the **Object** class.

Aditya now decides to place classes in a **max-heap**. All shown arrows below denote a true superclass-subclass relationship (for example, Snapp is a superclass of Wert and Laip), but **other superclass-subclass**

relationships may exist.



Using the heap above, determine whether each statement will *Never Error*, *Always Error*, or *Sometimes Error* (more information is needed to determine behavior). Assume that each class contains a constructor with no arguments, and that each class has its own unique `toString` method.

(e) (1 point) `Glorp g = new Bram();`

☒ Never Error

☐ Always Error

☐ Sometimes Error

The max-heap property says that each node is greater than all of its children. (see note)

Therefore, we know that Glorp is greater than Bram. In other words, Bram **is-a** Glorp. Therefore, a Bram can go in a Glorp memory box, and this assignment never errors.

To replace with more intuitive terms: Substitute Glorp with Dog, and substitute Bram with Poodle. Then, we can say Poodle **is-a** Dog. Therefore, `Dog d = new Poodle();` will never error, because a Poodle can go in a Dog memory box.

Note: Technically, the max-heap property says that each node is greater than *or equal to* all of its children. But in this question, we're dealing with strict inequalities; there's no notion of different classes being equal, and there are no duplicates in the heap. Therefore, we don't have to worry about equality in these questions.

(f) (1 point) `Foo f = new Wert();`

☐ Never Error

☐ Always Error

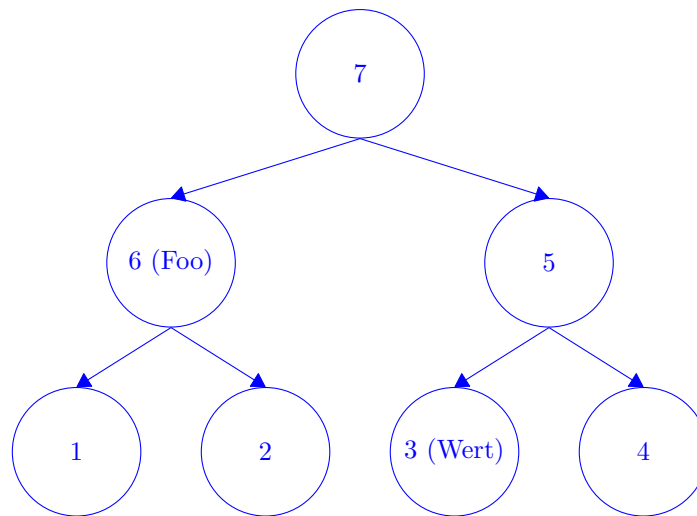
☒ Sometimes Error

From the heap diagram, there is no way to determine if `Foo > Wert` is true or false. The question says that other superclass-subclass relationships may exist, so it's possible that Wert extends Foo, and it's just not shown in the heap. On the other hand, it's possible that there is no inheritance relationship between Foo and Wert.

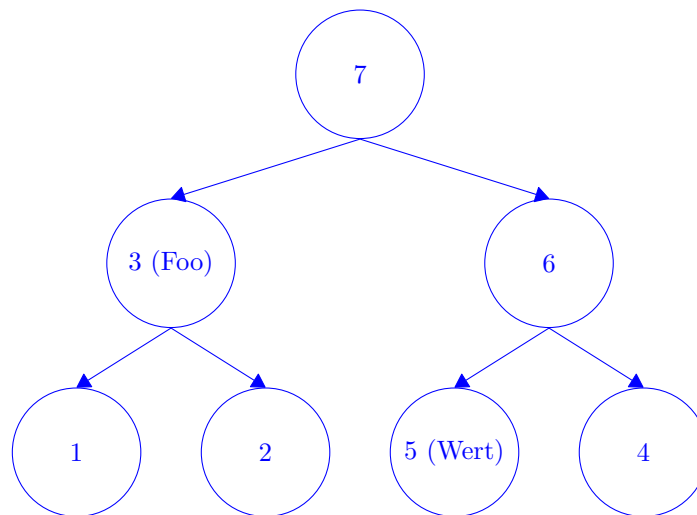
Because we don't know the inheritance relationship between Foo and Wert, we don't know what this assignment will do. It could succeed (e.g. if `Foo > Wert`), but it could also fail (e.g. if Foo and Wert have no inheritance relationship). Therefore, the statement sometimes errors.

To give a concrete example of why Foo and Wert cannot be compared, consider these two max-heaps of integers. Notice that in the first heap, the Foo node is larger than the Wert node. However, in the other heap, the Foo node is smaller than the Wert node.

First heap, Foo larger than Wert:



Second heap, Foo smaller than Wert:



(g) (1.5 points) What is the behavior of the code below?

```
Glorp g = new Snapp();
((Wert) g).toString();
```

- ☐ Glorp.toString is always called
 ☒ Always Error
 ☐ Wert.toString is always called
 ☐ Not Enough Information

From the heap, we can see that `Glorp > Snapp`, i.e. `Snapp` **is-a** `Glorp`. Therefore, a `Snapp` can go in a `Glorp` memory box, so the first line of code never errors. The reasoning is the same as part (e), so check there for a more detailed explanation.

`g` now has static type `Glorp`, and dynamic type `Snapp`.

We first act as the compiler and look at the second line of code. The static type of `g` is `Glorp`, but the cast is telling the compiler to treat `g` like a `Wert` instead, so we'll pretend like `g` has static type `Wert`. The question says to assume that every class has a `toString` method, so the compiler accepts this line of code (`Werts` have a `toString`, so we are good).

Next, we act as the runtime and look at the second line of code again. The dynamic type of `g` is `Snapp`. However, this contradicts the programmer's cast, which said that `g` is actually a `Wert`. Because the programmer's cast was not actually true (when compared to the object's runtime type), we encounter a runtime error.

If this explanation didn't make sense, we can substitute some more intuitive names. Let's substitute `Glorp` for `Animal`, and substitute `Snapp` for `Dog`, and `Wert` for `ShowDog`, so that the code becomes:

```
Animal g = new Dog();
((ShowDog) g).toString();
```

The first line is fine. Dogs can go into `Animal` memory boxes, because a `Dog` **is-an** `Animal`.

We first act as the compiler and look at the second line of code. `g` has static type `Animal`, but the programmer is telling us to pretend like `g` has static type `ShowDog`. Since `ShowDog` has a `toString` method (per the question assumption), the compiler accepts this line of code.

Next, we act as the runtime and look at the second line of code again. The dynamic type of `g` is `Dog`. However, this contradicts the programmer's cast, which said to treat `g` like a `ShowDog`. Because the programmer's cast was not actually true (when compared to the object's runtime type), we encounter a runtime error.

This runtime error is fairly similar to the first example presented in lecture, where we said that this results in a runtime error:

```
Poodle p = (Poodle) new Dog();
```

The compiler thinks this is fine, because the left-hand-side and right-hand-side both have static type `Poodle`. However, at runtime, the right-hand-side has dynamic type `Dog`, but the programmer is trying to pretend like the right-hand-side is a `Poodle`. This results in a runtime error.

Note: `Wert.toString` is just a common Java shorthand for "the `toString` method of the `Wert` class." It has nothing to do with static methods. `Wert.toString` not an actual piece of code that we're trying to run; we're actually trying to run `g.toString()`, and the question is whether this calls `Wert`'s `toString` method (abbreviated as `Wert.toString`) or `Glorp`'s `toString` method (abbreviated as `Glorp.toString`).

(h) (1.5 points) What is the behavior of the code below?

```
Laip l = new Wert();
((Snapp) l).toString();
```

- | | |
|---|---|
| ☐ <code>Laip.toString</code> is always called | ☐ Always Error |
| ☐ <code>Wert.toString</code> is always called | ☒ Not Enough Information |

The solution is very similar to subpart (g).

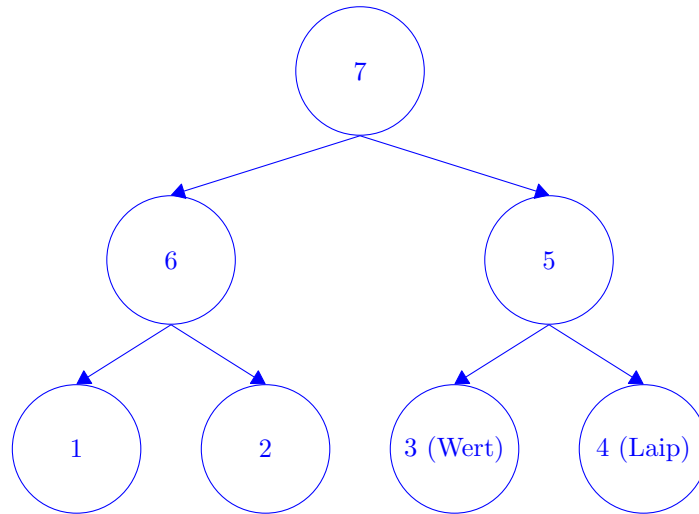
From the heap diagram, there is no way to determine if `Laip > Wert` is true or false. The question says that other superclass-subclass relationships may exist, so it's possible that `Wert` extends `Laip`, and it's

just not shown in the heap. On the other hand, it's possible that there is no inheritance relationship between Laip and Wert.

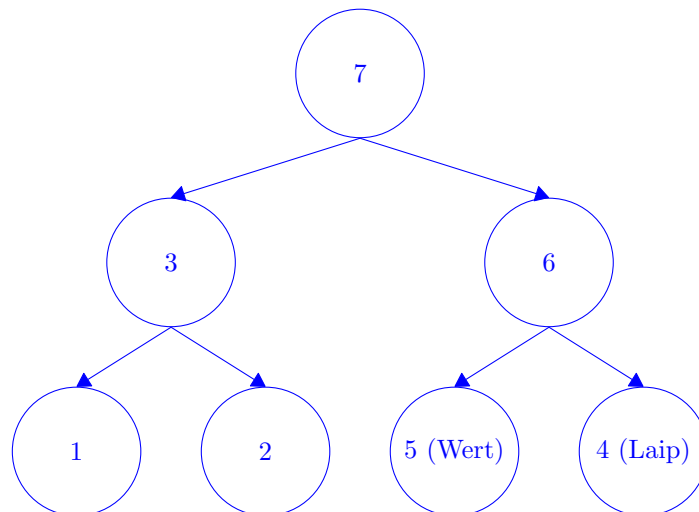
Because we don't know the inheritance relationship between Laip and Wert, we don't know what this assignment will do. It could succeed (e.g. if $\text{Laip} > \text{Wert}$), but it could also fail (if Laip and Wert have no inheritance relationship). Therefore, the statement sometimes errors.

To give a concrete example of why Laip and Wert cannot be compared, consider these two max-heaps of integers. Notice that in the first heap, the Laip node is larger than the Wert node. However, in the other heap, the Laip node is smaller than the Wert node.

First heap, Laip larger than Wert:



Second heap, Foo smaller than Wert:



3 The Golden Apples of Yokeyrin

(21 points)

Anirudh the Atlas-bearing adventurer has finally reached Room 100 of Yokeyrin's cave to retrieve his real target: the Golden Apples of Yokeyrin. The goal is to give the Golden Apples to Heracles, who is currently in Room 1. They want to meet anywhere within the cave so that Anirudh can give Heracles the Golden Apples as fast as possible.

Fill out the `leastTime` method, which is defined as follows:

- Input: `Map<Integer, Set<Edge>> adj`, which maps each room (1 through 100) to a set of edges. Each `Edge` contains the room to which the `Edge` leads and the time needed to traverse the `Edge`.
- Output: The minimum time needed for Anirudh and Heracles to meet in the same room, if Anirudh and Heracles start at rooms 100 and 1, respectively.
- Heracles and Anirudh move simultaneously through the cave, and are allowed to wait in a room (without traversing an edge). Heracles and Anirudh are NOT allowed to meet in the middle of an edge.
- There is guaranteed to be at least one room where Anirudh and Heracles can meet.

```
public class State {

    public int room;
    public int time;

    public State(int room, int time) {
        this.room = room;
        this.time = time;
    }
}

public class Edge {

    public int dest; // Destination, i.e. room where the Edge leads.
    public int weight; // Time needed to traverse the Edge.

    public Edge(int dest, int weight) {
        this.dest = dest;
        this.weight = weight;
    }
}
```

(a) (13 points) Fill out the following helper methods:

```

public class GoldenApple {

    private class StateComparator implements Comparator<State> {

        @Override
        public int compare(State o1, State o2) {

            return o1.time - o2.time;
        }
    }

    private Map<Integer, Integer> dijkstras(int source, Map<Integer, List<Edge>> adj) {

        Map<Integer, Integer> distTo = new HashMap<>();

        PriorityQueue<State> states = new MinHeap<>(new StateComparator());
        states.add(new State(source, 0));

        while (!states.isEmpty()) {

            State curr = states.poll();

            if (!distTo.containsKey(curr.room)) {

                distTo.put(curr.room, curr.time);

                for (Edge e : adj.get(curr.room)) {

                    State nextRoom = new State(e.dest, curr.time + e.weight);

                    states.add(nextRoom);
                }
            }
        }
        return distTo;
    }
}

```

Explanation (StateComparator):

The method name is `dijkstras`, which is a strong hint that we should probably be implementing Dijkstra's algorithm in this question.

We see that we are given `PriorityQueue<State> states`. This will be our fringe, which keeps track of the nodes that we've visited so far.

The provided `PriorityQueue` takes in `States` instead of just integers (representing rooms). The provided code shows that a `State` contains not just the room, but also the time it takes to reach that room. The reason we're using `States` (with room and time) instead of integers (just the room) is because we need to do additional book-keeping to help us keep track of how long it takes to reach each room.

If we have many `States` on the fringe, how do we know which one to take off the fringe first? Remember that Dijkstra's algorithm visits the nodes in best-first order. This means that the next node we visit from the fringe should be the closest node on the fringe, i.e. the node with the lowest cost/time so far. (In this question, cost and time are used interchangeably, since the costs in the problem are the times it takes to reach each room.)

The `PriorityQueue` fringe has no idea how we want to compare `States`, but luckily, there's a `StateComparator` class that we can fill in. This `StateComparator` should take in two states and compare them by their cost/time, since we want the fringe to visit nodes in order of cost/time.

`return o1.time - o2.time` is a handy trick we saw in lecture. It returns a negative number if `o1 < o2`, zero if `o1 == o2`, and a positive number if `o1 > o2`. This is what the `Comparator` interface expects.

Now that we've created the `StateComparator` class, we need to create a new `StateComparator` object and pass it into the `MinHeap`, so that the `MinHeap` can use the new `StateComparator` object to compare `State` objects and return the next node to explore.

Explanation (Dijkstra's Algorithm):

With the `StateComparator` complete, the rest of the code is fairly similar to Dijkstra's algorithm as seen in lecture. On each iteration, we call `states.poll()` to take the next-nearest node off the fringe. We update the `distTo` map to log that the best-known cost to `curr.room` is `curr.time`.

Then, for all of the current node's outgoing edges, we need to relax all those outgoing edges. In the standard formulation of Dijkstra's, we achieved this with the `changePriority` operation on the `MinHeap`. For example, if we have an edge from Room 75 to Room 89, we'd change the priority of Room 89 if `[current cost to Room 75] + [edge cost from Room 75 to Room 89]` is better than the currently best-known cost to Room 89.

In this slightly modified version of Dijkstra's algorithm, we don't have the `changePriority` operation, so we just add a new `State` object onto the fringe, with the next room (`e.dest`, e.g. Room 89), and the new potential cost to that next room (`curr.time`, e.g. current cost to Room 75, plus `e.weight`, e.g. edge cost from Room 75 to Room 89).

If this new potential cost is better, then the new `State` we built will be popped off the fringe earlier than any other state for Room 89. Any other states for Room 89 are worse ways to reach Room 89, so they will be popped off the fringe later. By the time those worse Room 89 states are popped off the fringe, we already visited Room 89, so the `if (!distTo.containsKey(curr.room))` check will ensure that we don't visit Room 89 a second time.

If the new potential cost is worse, then the new `State` we built will be popped off the fringe later. In other words, some other Room 89 state will be popped off the fringe first. By the time our new `State` (with worse po-

tential cost) is popped off the fringe, we already visited Room 89, so the `if (!distTo.containsKey(curr.room))` check will ensure that we don't visit Room 89 a second time.

Alternate Solutions (Blanks 1 and 2):

`Comparable<State>` instead of `Comparator<State>` is **incorrect**. We are overriding the `compare(State o1, State o2)` method, and the `compare` method only exists in the `Comparator` interface. The `Comparable` interface instead has a method like `compareTo(State other)`, so if we said `implements Comparable<State>` instead, we would be overriding the wrong method.

`extends` instead of `implements` is **incorrect**. `Comparator` is an interface, so we must use the `implements` keyword.

`Comparator<E>` or `Comparator<T>` or any other placeholder word instead of `Comparator<State>`, is **incorrect**. The generic `Comparator<T>` interface has a generic method `compare(T o1, T o2)`, and when you create a specific `Comparator`, you need to replace the `T` placeholder to indicate what specific thing you're trying to compare. In this case, the provided function signature `compare(State o1, State o2)` shows that the thing we want to compare is `States`, so we need to say `Comparator<State>` in order to match the type in the method signature.

Alternate Solutions (Blank 3):

`return Integer.compare(o1.time, o2.time)` instead of `return o1.time - o2.time` is correct.

`return o1.time.compareTo(o2.time)` is **incorrect**, because `o1.time` is of type `int`, and primitive types don't have methods. For example, you can't say `5.compareTo(6)` because `5` is not an object and it has no methods.

Nothing else really works here. Here are some examples of wrong answers:

- `return o1.room - o2.room`
- `return o1.room.compareTo(o2.room)`
- `return Math.abs(o1.time - o2.time)`
- `return Math.min(o1.time, o2.time)`
- `return o1.compareTo(o2)`
- `return o2.time - o1.time`
- `return o1.time > o2.time`
- `return o1.time == o2.time`
- `return o1.room == o2.room`
- `return (o1.time || o2.time)`
- `return o1 - o2`
- Any answer that doesn't include the word `time`

Alternate Solutions (Blank 4):

Due to a bug on the reference card,

```
new PriorityQueue<>(new StateComparator());
and
new MinHeap<>(new StateComparator());
```

were both accepted as correct answers.

`new MinHeap<State>(new StateComparator())` or `new PriorityQueue<State>(new StateComparator())` is also correct (i.e. adding the word `State` inside the angled brackets is fine).

`new MinHeap<>(StateComparator)` and `new MinHeap<>(StateComparator())` are **incorrect**. The `MinHeap` constructor expects a `StateComparator` object, but no such object exists in the starter code. Therefore, you must explicitly create a new `StateComparator` object with the `new` keyword. It might be more intuitive to see the solution as two separate lines (though the blanks on the exam forced you to combine them into one line):

```
Comparator<State> c = new StateComparator();
PriorityQueue<State> states = new MinHeap<>(c);
```

Alternate Solutions (Blank 5):

No alternate solutions that we know of.

Alternate Solutions (Blank 6):

`distTo.get(curr.room)` and `curr.time` are equivalent (because we did `distTo.put(curr.room, curr.time)` on Blank 5), so both of these answers were considered correct:

```
new State(e.dest, distTo.get(curr.room) + e.weight)
new State(e.dest, curr.time + e.weight)
```

Alternate Solutions (Blank 7):

Instead of assigning `nextRoom` on Blank 6 and then using it on Blank 7, you could do everything on Blank 7, and leave Blank 6 empty:

```
states.add(new State(e.dest, curr.time + e.weight))
```

or

```
states.add(new State(e.dest, distTo.get(curr.room) + e.weight))
```

Note that for this alternate solution to be correct, you'd have to do something harmless on Blank 6, e.g. leave it empty, or do a harmless dummy assignment like `State nextRoom = new State(0, 0)`. Then, you'd have to never use `nextRoom` in Blank 7. Writing something that crashes the program on Line 6, e.g. `State nextRoom = 5`, would not work.

The `State` and `Edge` classes are reprinted below for your convenience.

```
public class State {
    public int room;
    public int time;
    ...
}
```



```

}

public class Edge {
    public int dest; // Destination, i.e. room where the Edge leads.
    public int weight; // Time needed to traverse the Edge.
    ...
}

```

(b) (8 points) Fill out the implementation of `leastTime` below:

```

public class GoldenApple {
    // ... continued from previous page

    public int leastTime(Map<Integer, List<Edge>> adj) {

        Map<Integer, Integer> herc = dijkstras(1, adj);

        Map<Integer, Integer> ani = dijkstras(100, adj);

        int lowestTime = Integer.MAX_VALUE;

        for (int i = 1; i <= 100; i++) {

            if (herc.containsKey(i) && ani.containsKey(i)) {

                int currTime = Math.max(herc.get(i), ani.get(i));

                lowestTime = Math.min(lowestTime, currTime);
            }
        }

        return lowestTime;
    }
}

```

Explanation:

The `dijkstras` helper method tells us the shortest paths tree from a given source node. In other words, it can tell us the shortest time to reach every other room from a given start room.

We have this helper method that returns a `Map<Integer, Integer>`, and we have two blanks where we need to assign `Map<Integer, Integer>`, so a natural way to fill in those blanks is to call the `dijkstras` helper method.

Two natural source nodes from which we'd want the shortest paths tree are Room 1 (Heracles's starting location) and Room 100 (Anirudh's starting location).

The provided for-loop indicates that we should be doing some check for each of the 100 rooms.

First, we should check that room `i` is actually a room that both Heracles and Anirudh can reach. (The question says that there is at least one room where they can both meet, but it does not guarantee that Heracles and Anirudh can each reach all of the rooms.) The line `if (herc.containsKey(i) && ani.containsKey(i))` checks that both Heracles and Anirudh can even reach this room.

Note that if we omitted these checks, then `herc.get(i)` or `ani.get(i)` could return `null` inside the if-case, and calling `Math.max(herc.get(i), ani.get(i))` would crash the program (you can't ask for the max of `null` and 5).

Now that we know that Heracles and Anirudh can both reach room `i`, how long does it take them to meet at room `i`? As an example, let's say Heracles takes 25 time steps to reach this room, and Anirudh takes 35 time steps to reach this room. The question says that Heracles and Anirudh can move simultaneously, and can wait in a room. This means that if Heracles should travel to room `i` in 25 time steps, and then wait there. At the same time, Anirudh should travel to room `i` in 35 time steps. They will meet at room `i` when Anirudh arrives at time step 35.

More generally, if Heracles takes `herc.get(i)` time to reach room `i`, and Anirudh takes `ani.get(i)` time to reach room `i`, it will take `Math.max(herc.get(i), ani.get(i))` for them to meet at this room. The person closer to the room will get there first and wait for the other person to get there.

The loop computes `currTime` once for every possible meeting room. Finally, we just need to find the room with the lowest value of `currTime`, which is the room where Heracles and Anirudh can meet in the fastest time. `lowestTime = Math.min(lowestTime, currTime)` keeps track of the best meeting time we've found so far. After checking all 100 rooms, we `return lowestTime`, the best meeting time we found, and we are done.

Alternate Solutions (Blanks 1 and 2):

Swapping Blanks 1 and 2 is fine, e.g. you can put `dijkstras(100, adj)` in Blank 1 and `dijkstras(1, adj)` in Blank 2.

The variable names don't really make sense if you do this, since `herc` starts in Room 1 but you assigned `herc = dijkstras(100, adj)`, but the resulting code is still correct.

Alternate Solutions (Blanks 3 and 4):

`if(herc.get(i) && ani.get(i))` is **incorrect**. The `get` method returns an integer or `null`, but integers and `null` are not Boolean values in Java, so this causes a type error. (In other languages like C, this might be fine, though.)

`herc.get(i) < lowestTime && ani.get(i) < lowestTime`, and similar attempts to compare values in the if-statement, are **incorrect**, because the `get` calls might return `null`, and you can't compare `null` to an integer. As mentioned in the explanation, we are only guaranteed that Heracles and Anirudh can meet in at least one room; we are not guaranteed that they can meet in every room.

Alternate Solutions (Blank 5):

`herc.get(i) + ani.get(i)` was worth partial credit, if you forgot that Heracles and Anirudh can move simultaneously. This solution computes the lowest time if you assume that their movements take turns, e.g. it takes 2 time steps for Heracles and Anirudh to each move once. However, since they can move simultaneously, you really want the max of the two times, not their sum.

Alternate Solutions (Blank 6):

No alternate solutions that we know of.

4 Sorting

(18 points)

Consider running in-place Heapsort on the array $[7, 2, 2, 4, 5, 8, 9]$.

(a) (1.5 points) What is the state of the array immediately after completing bottom-up heapification?

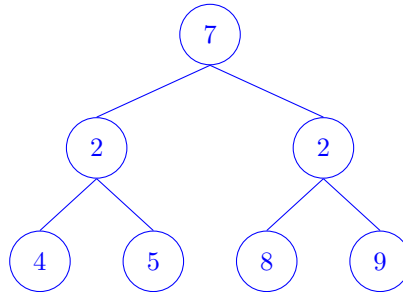
☐ $[9, 8, 5, 4, 7, 2, 2]$

☒ $[9, 5, 8, 4, 2, 7, 2]$

☐ $[2, 4, 2, 7, 5, 8, 9]$

☐ $[9, 5, 8, 2, 4, 2, 7]$

First, we'll draw the heap diagram corresponding to this array representation:

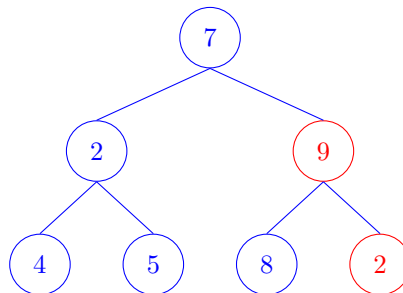


We now need to perform bottom-up heapification by sinking every node in reverse-level order (right-to-left, bottom-to-top). In other words, we need to sink 9, 8, 5, 4, 2, 2, 7, in that order.

Reminder: In-place heapsort, as presented in lecture, should result in a max-heap, not a min-heap.

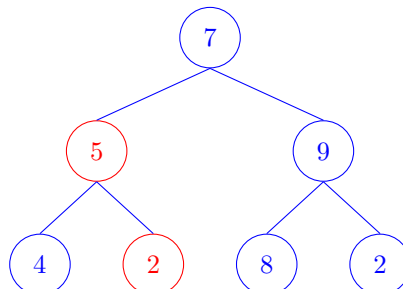
Sinking 9, 8, 5, 4 doesn't change anything.

Next, we sink the right 2, which causes it to swap with 9.

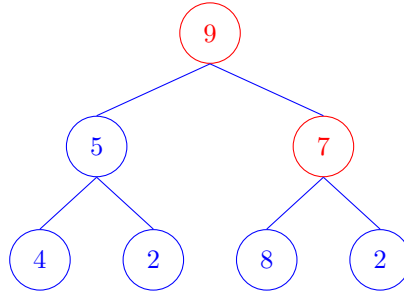


Reminder: When we sink a node, we should swap with the “better” child (larger child in a max-heap, smaller child in a min-heap). If you tried to swap 2 and 8 instead of 2 and 9, the resulting max-heap would be invalid (8 cannot be above 9).

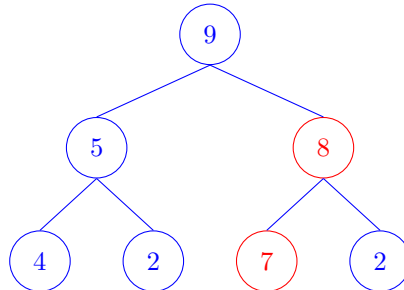
Next, we sink the left 2, which causes it to swap with 5.



Next, we sink 7, which first causes it to swap with 9...



...and then causes 7 to swap again with 8 to complete the sinking.



Now that we're done, we read off the values on the heap in level order (left-to-right, top-to-bottom) to get the array representation of the heap: [9, 5, 8, 4, 2, 7, 2], and we are done.

- (b) (1.5 points) What is the state of the array after exactly one iteration of Heapsort (moving one item to its correct spot and any resulting heap operations)?

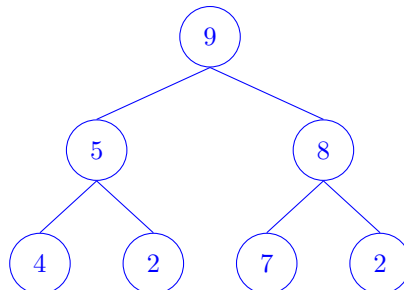
☐ [5, 4, 8, 2, 2, 7, 9]

☐ [2, 5, 8, 4, 2, 7, 9]

☐ [8, 7, 4, 5, 2, 2, 9]

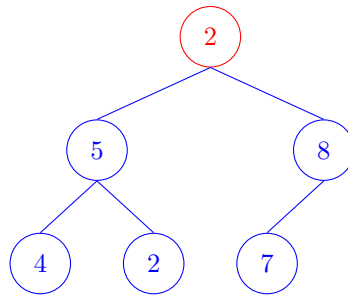
☒ [8, 5, 7, 4, 2, 2, 9]

From the previous subpart, the array after bottom-up heapification looks like this:

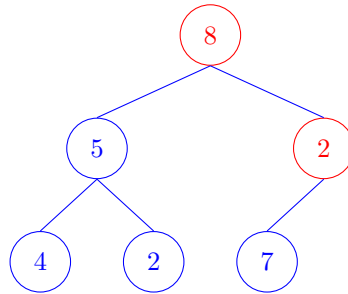


In each iteration of heapsort, we delete the largest item from the heap, and put that item at the back of the array.

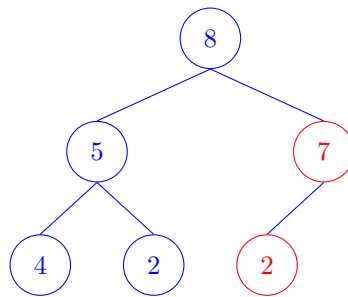
To delete the largest item, we delete the root (9), and we swap the last (bottom-right) item (2) into the root:



Then, we sink 2 to fix the max-heap. This first causes 2 and 8 to swap...



...then, it causes 2 and 7 to swap, completing the sink operation.



We can now read off the values on the heap in level order (left-to-right, top-to-bottom) to get the array representation of the heap: $[8, 5, 7, 4, 2, 2]$. The deleted item (9) is added to the end of the array, so our final answer is $[8, 5, 7, 4, 2, 2, 9]$, and we are done.

- (c) (1.5 points) Consider running MSD radix sort on the array $[123, 312, 279, 189, 423, 275]$.

What is the state of the array after the first two digits have been processed?

- ☐ $[123, 189, 275, 279, 312, 423]$
☒ $[123, 189, 279, 275, 312, 423]$
☐ $[312, 123, 423, 275, 279, 189]$
☐ $[312, 123, 423, 279, 275, 189]$

Reminder: MSD radix sort starts with the most-significant (left-most) digit, and works its way from left to right. MSD radix sort is stable, which means that items considered equal (based on the current digit being compared) don't cross over.

First, we sort by the left-most digit, which means that all the numbers starting with 1 come first, followed by all numbers starting with 2, and so on:

$[123, 189]$ $[279, 275]$ $[312]$ $[423]$

Note that 279 and 275 are considered equal on this pass, because they both start with 2. Since MSD radix sort is stable, the relative order of these items (279 before 275) is preserved after the sort.

At this point, we have four smaller sub-problems. Within each sub-problem, we need to sort by the next digit (the middle digit):

[123, 189] [279, 275] [312] [423]

By coincidence, sorting by the middle digit doesn't change anything. For example, in the first sub-problem, **123** coming before **189** is correct, since 2 comes before 8. Likewise, in the second subproblem, **279** and **275** are considered equal (same middle digit), so they keep their same relative ordering (because each pass of MSD radix sort is stable).

Thus, our final answer is [123, 189, 279, 275, 312, 423], and we are done.

- (d) (1.5 points) Consider running LSD radix sort on the array [123, 312, 279, 189, 423, 275].

What is the state of the array after the last two digits have been processed?

- | | |
|---|--|
| ☐ [123, 189, 275, 279, 312, 423] | ☐ [123, 189, 279, 275, 312, 423] |
| ☒ [312, 123, 423, 275, 279, 189] | ☐ [312, 123, 423, 279, 275, 189] |

Reminder: LSD radix sort starts with the least-significant (right-most) digit, and works its way from right to left. LSD radix sort is stable, which means that items considered equal (based on the current digit being compared) don't cross over.

First, we sort by the right-most digit, which means that all the numbers ending with 1 come first, followed by all numbers ending with 2, and so on:

[312, 123, 423, 275, 279, 189]

Note that **123** and **423** are considered equal on this pass, because they both end with 3. Since LSD radix sort is stable, the relative order of these items (123 before 423) is preserved after the sort.

Unlike MSD radix sort, we don't create separate sub-problems in LSD sort. On the next pass, we sort the entire array again, but this time using the middle digit:

[312, 123, 423, 275, 279, 189]

Again, by coincidence, sorting by the middle digit doesn't change anything. Note that 123 and 423 are considered equal (same middle digit), so they keep their same relative ordering (because each pass of LSD radix sort is stable).

Thus, our final answer is [312, 123, 423, 275, 279, 189], and we are done.

For the remaining subparts, find each modified sort's best- and worst-case runtimes to sort N **distinct** items, in terms of N . If a runtime is infinite, write $\Theta(\infty)$.

Consider an extension of Mergesort, where the array is partitioned into k subarrays during each recursive step ($k = 2$ in standard Mergesort). The merging algorithm is extended to handle k subarrays:

1. During the merge step, maintain k pointers, initialized to the start of each subarray.
2. Look through all pointed elements (ignoring any pointers that are already past the end of their subarray), and select the smallest pointed element.
3. Place that item into the merged array, and advance the pointer of the corresponding subarray.
4. Repeat Steps 2 and 3 until all k pointers are past the end of their respective subarrays.

(e) (3 points) What is the runtime behavior for $k = 3$?

Best Case:

$$\Theta(N \log N)$$

Worst Case:

$$\Theta(N \log N)$$

At a high level, this algorithm does the same thing as mergesort, but the base of the logarithm changes, and the logarithm base is a constant factor we can ignore (look up change-of-base formula to see why).

To prove this more rigorously, we can analyze the runtime of the merge step. Then, we can draw out a recursive tree to see how much total work is done across all the recursive calls.

Merge step runtime analysis:

Consider a single merge operation, where we have three sorted subarrays, each of size $N/3$, and we need to merge them into a single sorted array of size N . How long does this step take?

1. We have 3 pointers, initialized to the start of each subarray. Initializing 3 pointers takes $\Theta(1)$ time.
2. Look through all pointed elements, and select the smallest one. There are 3 pointed elements (one in each subarray), so we just need to select the smallest of 3 numbers. Therefore, this step takes $\Theta(1)$ time.

Note that k is fixed to a constant number (3) in this algorithm. We are not trying to analyze what happens when k gets large. That's why the runtime for checking the smallest of $k = 3$ numbers is $\Theta(1)$ time. Regardless of how big N is, this step is always checking just 3 numbers.

3. Place that item into the merged array, and advance the pointer of the corresponding subarray. This step takes $\Theta(1)$ time, since we copy one item and advance one pointer.

4. Repeat Steps 2 and 3 until all k pointers are past the end of their respective subarrays.

Note that Steps 2 and 3 are each $\Theta(1)$ time, so each repeat of Steps 2 and 3 takes $\Theta(1)$ time.

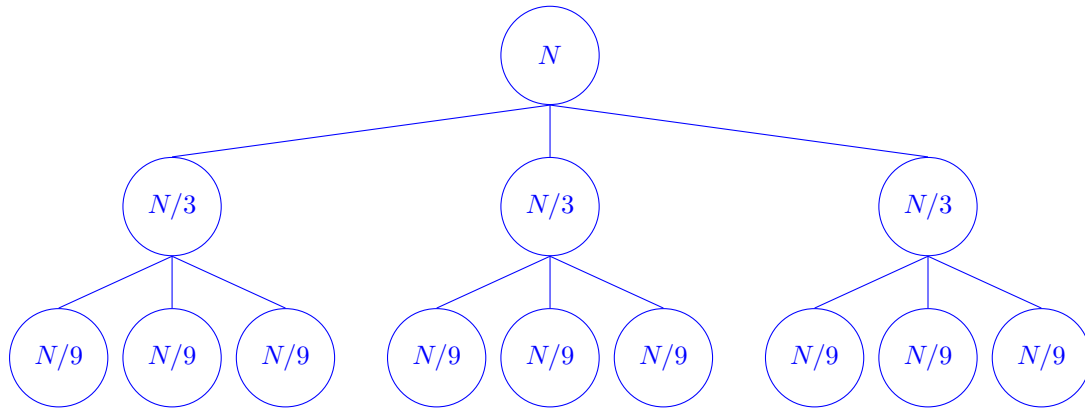
How many times do we repeat Steps 2 and 3? Each time we run Steps 2 and 3, we get a single new item placed into the merged array. We eventually need all N items placed into the merged array, so we need to repeat N times.

If we repeat the steps N times, and each repeat takes $\Theta(1)$ time, the total runtime of this merge is $\Theta(N)$.

This analysis turns out to be the same as the standard $k = 2$ mergesort, where merging is also a $\Theta(N)$ operation. The key observation is that both $k = 2$ and $k = 3$ don't depend on N . In both cases, adding an item into the merged array is constant time (either find the minimum of 2 items, or the minimum of 3 items, neither of which depend on N), and you have to add a total of N items into the merged array.

Recursive call runtime analysis:

We can draw a tree to represent all the recursive calls. Each node represents a recursive call to mergesort, e.g. a node with $N/3$ means that we call mergesort with a subarray of size $N/3$. The edges tell us which recursive calls were made, e.g. an edge connecting N to $N/3$ means that mergesorting a size N array required a recursive call to mergesort a size $N/3$ array.



The $N/9$ nodes would each have three $N/27$ children nodes, and so on.

In this case, the value in the node tells us how much work is done in that specific recursive call. For example, consider the root node N . In this call to mergesort, we have to take three subarrays of size $N/3$ and merge them into an array of size N , and we saw that this takes N time.

Likewise, consider any one of the $N/3$ nodes. In this call to mergesort, we have to take three subarrays of size $N/9$ and merge them into an array of size $N/3$. Merging into an array of size $N/3$ takes $N/3$ time (since we know that merging is linear-time).

Because each node tells us how much work is done in that recursive call, we can sum up the values in every node, and that should tell us the total amount of work done.

There are several ways to sum up all the values in every node, but the quickest one is to notice that the total amount of work done at each level is constant:

At the zeroth level, there is one node with N work done, for a total of N work.

At the first level, there are three nodes with $N/3$ work done per node, for a total of N work.

At the second level, there are nine nodes with $N/9$ work done per node, for a total of N work.

At the third level (not drawn), there are 27 nodes with $N/27$ work done per node, for a total of N work.

So to get the total amount of work, we just need to multiply N (work done per level) with the number of levels.

How many levels are there? This tree has:

N at the zeroth level.

$\frac{N}{3^1} = N/3$ at the first level.

$\frac{N}{3^2} = N/9$ at the second level.

$\frac{N}{3^3} = N/27$ at the third level (not drawn).

$\frac{N}{3^4} = N/81$ at the fourth level (not drawn).

$\frac{N}{3^5} = N/243$ at the fifth level (not drawn).

$\frac{N}{3^b}$ at the b th level, in general.

The base case of mergesort is when we have an array with 1 item, so the last level is the level where the nodes have 1 in them.

To figure out which level has the nodes with 1 in them, we could write an equation like this:

$$\frac{N}{3^b} = 1$$

Rearranging terms:

$$N = 3^b$$

$$b = \log_3(N)$$

This tells that the $\log_3(N)$ th level is the one with the base case, and that there are $\log_3(N)$ levels in total.

Another intuitive way to find the total number of levels is to realize that at each level, we divide N by 3. To figure out how many levels we need before reaching 1, we have to answer the question: How many times do you need to divide N by 3 before you reach 1? The answer to this question is exactly the definition of $\log_3(N)$.

To finish the problem: We do N work at every level, and there are $\log_3(N)$ levels in total. Therefore, the total amount of work done is $N \log_3(N)$. The base of the logarithm is a multiplicative constant that we can drop (look up change-of-base formula to see why), so $N \log_3(N)$ has order-of-growth $\Theta(N \log N)$, and we are done.

Note on best-case and worst-case:

The best-case and worst-case runtimes are both $\Theta(N \log N)$ for a similar reason as merge sort. No matter what array you provide as input, mergesort always takes $\Theta(N \log N)$ time. The recursive tree always looks the same (arrays split into equally-sized subarrays), and the merging step always takes $\Theta(N)$ time (you always have to copy N items into the merged array).

As a comparison, quicksort's best-case and worst-case runtime are different because quicksort can split arrays into unequal-sized subarrays. For example, if you pick a bad pivot, everything might end up in the right subarray, leaving the left subarray empty. This does not happen in mergesort.

- (f) (3 points) What is the runtime behavior for $k = N$?

Best Case:

$$\Theta(N^2)$$

Worst Case:

$$\Theta(N^2)$$

At a high level, this algorithm does the same thing as selection sort, which takes $\Theta(N^2)$ time.

To prove this more rigorously, we first observe that this sort is not recursive, and it finishes in a single merge step. Then, we can analyze the runtime of that single merge step.

Key observation: It's Not Recursive

The first key observation is that by setting $k = N$, we end up splitting a size- N array into N subarrays. Every subarray is of size 1!

Sorting an array of size 1 is trivial (base case). Therefore, this algorithm is not really recursive: We split the array into N subarrays, each of size 1. Then, we perform just a single merge step to combine all the subarrays into a single size- N array.

Merge step runtime analysis:

Now that we know it's not recursive, we just need to figure out how long the single merge step takes.

Remember, we have split the array into N subarrays, each of size 1. The goal is to merge everything back into a single size- N array.

1. We have $k = N$ pointers, initialized to the start of each subarray. Initializing these pointers takes $\Theta(N)$ time.
2. Look through all pointed elements, and select the smallest one. There are $N = k$ pointed elements, and we need to look through all of them to find the smallest element. This step takes $\Theta(N)$ time.
3. Place that item into the merged array, and advance the pointer of the corresponding subarray. This step takes $\Theta(1)$ time, since we copy one item and advance one pointer.
4. Repeat Steps 2 and 3 until all k pointers are past the end of their respective subarrays.

Note that a single repeat of Steps 2 and 3 takes $\Theta(N)$ time. (It's the sum of a $\Theta(N)$ operation, followed by a $\Theta(1)$ operation.)

How many times do we repeat Steps 2 and 3? Each time we repeat, we get a single new item placed into the merged array. We eventually need all N items placed into the merged array, so we need to repeat N times.

If we repeat the steps N times, and each repeat takes $\Theta(N)$ time, the total runtime of the merge is $\Theta(N^2)$.

Observation: It's Just Selection Sort

In each repeat, we find the smallest item in the remaining subarrays, and copy that item into the merged array. This is exactly the same as what selection sort does, and selection sort runs in $\Theta(N^2)$ time.

If you wanted to be a bit more rigorous, you can notice that each time you repeat, one of the pointers falls off the end of the subarray. This is because each subarray is size-1, so when you copy the item from that subarray, its pointer falls off the end, and you don't have to consider that pointer again.

Thus, on the first repeat, you have $k = N$ pointers to consider (one for each subarray).

On the next repeat, you have $N - 1$ pointers to consider, because one of them has fallen off the end of its subarray.

On the next repeat, you have $N - 2$ pointers to consider.

On the very last repeat, you'll have just 1 pointer left to consider. (All other items have been copied into the merged array, and only a single pointer has not fallen off the end yet.)

Thus, the total work done is N on the first repeat, $N - 1$ on the next repeat, $N - 2$ on the next repeat, and so on, all the way to 1 on the last repeat. This sum is $N + (N - 1) + (N - 2) + \dots + 3 + 2 + 1$, which we know is $\Theta(N^2)$. This is also the same sum that appears in our selection sort runtime analysis.

Consider a variant of Quicksort. In this variant, the pivot is chosen as the left-most element of the array. The array is partitioned into two subarrays:

- The left subarray contains all elements strictly less than the pivot.
- The right subarray contains all elements greater than or equal to the pivot, including the pivot.

After partitioning, the pivot is placed in the right subarray. Then, the algorithm is recursively called on each subarray with more than one element.

- (g) (3 points) What is the runtime behavior if the pivot is always placed at the left-most position in the right subarray?

Best Case:

$$\Theta(N^2)$$

Worst Case:

$$\Theta(\infty)$$

Trying examples, and finding the worst case:

When asked to analyze an unfamiliar algorithm, a good first step is to run it with some sample inputs.

Let's try sorting $[1, 2, 3, 4, 5]$ with this algorithm.

The pivot is 1. The left subarray is empty. The right subarray has everything greater than or equal to the pivot, including the pivot: $[1, 2, 3, 4, 5]$. Notice that we place the pivot (1) in the left-most position of this subarray, as specified in the question.

We need to recursively sort the right subarray. The pivot is 1. The left subarray is empty. The right subarray has everything greater than or equal to the pivot, including the pivot: $[1, 2, 3, 4, 5]$. Again, we put the pivot (1) in the left-most position.

We need to recursively sort the right subarray. Do you notice anything interesting? On every step, we input array $[1, 2, 3, 4, 5]$, and the output is the subarray $[1, 2, 3, 4, 5]$, which we need to recursively sort again. No matter how many times we repeat the process, we aren't making any progress at all! This algorithm will infinite-loop forever.

This tells us that the worst-case runtime is an infinite loop. You can't do any worse than an infinite loop, so we know that this is definitely our worst case, so we are done.

Trying other examples, and spotting a pattern:

Let's try some other inputs to see if we can find some other interesting behavior.

Consider sorting $[2, 4, 1, 3, 5]$ with this algorithm.

The pivot is 2. The left subarray is $[1]$. The right subarray is $[2, 4, 3, 5]$. We place the pivot (2) in the left-most position of this subarray, as specified in the question.

The left subarray is trivially sorted: $[1]$.

Now we need to sort the right subarray: $[2, 4, 3, 5]$.

The pivot is 2. The left subarray is empty. The right subarray is [2, 4, 3, 5], with the pivot (2) in the left-most position.

Now we need to sort the right subarray: [2, 4, 3, 5]. This is exactly the same as what we were sorting before! It looks like this input array also runs into an infinite-loop.

What if we tried sorting [5, 1, 2, 3, 4] with this algorithm?

The pivot is 5. The left subarray is [1, 2, 3, 4]. The right subarray is [5].

We now recursively sort [1, 2, 3, 4]. The pivot is 1. The left subarray is empty. The right subarray is [1, 2, 3, 4], with the pivot (1) in the left-most position.

We now recursively sort [1, 2, 3, 4]. This is the same subarray again! It looks like this input array also runs into an infinite-loop.

If you try out a few more inputs, you'll see that almost all of them result in an infinite loop.

Now that we've tried lots of examples, we can start to spot patterns. More generally, the infinite loop always appears after the first iteration, for two reasons:

Reason 1: We place the pivot in the *left-most* position in the *right* subarray. Since we choose the left-most item as the pivot, this means that when we go to recursively sort the right subarray, we end up picking the same item as the pivot again.

As a concrete example: When we sorted [1, 2, 3, 4, 5], we chose 1 as the pivot. The right subarray was [1, 2, 3, 4, 5], and notice that 1 gets chosen as the pivot again (because it's placed in the left-most spot of the subarray).

Reason 2: The right subarray contains everything greater than or equal to the pivot, and we keep choosing the same pivot over and over again. Therefore, every single partition ends up putting everything in the right subarray (because everything is greater than the pivot).

As a concrete example: When we sorted [1, 2, 3, 4, 5], we kept choosing 1 as the pivot. Every time we choose 1 as the pivot, everything in the subarray is greater than 1, so everything ends up in the same subarray. The partitions never create smaller subarrays.

Finding the best case:

Is it possible that the best-case is also an infinite-loop, i.e. every single input causes an infinite loop? Or can we somehow avoid the invariants described above, to find a case where the input doesn't infinite-loop?

Note: You might be tempted to say that the best-case is constant-time, if you input a size-1 array. But remember that in asymptotic runtime analysis, our goal is to determine the behavior of the algorithm as N grows large. Here, N is the size of the input array, so we cannot use a size-1 input for asymptotic analysis.

From our reasoning above, it seems like every time we create a non-trivial right subarray, we get an infinite-loop, because the pivot ends up in the right subarray, and this same pivot is chosen over and over again.

What if we ensured that the right subarray is always trivial (size 0 or 1), and everything ends up in the left subarray?

Consider sorting an input like this: [5, 4, 3, 2, 1].

The pivot is 5. The left subarray is [4, 3, 2, 1], and the right subarray is [5] only.

We now need to recursively sort [4, 3, 2, 1]. The pivot is 4. The left subarray is [3, 2, 1], and the right subarray is [4] only.

We now need to recursively sort [3, 2, 1]. The pivot is 3. The left subarray is [2, 1], and the right subarray is [3] only.

We now need to recursively sort [2, 1]. The pivot is 2. The left subarray is [1], and the right subarray is [2].

At this point, every subproblem is of size 1, so we are done. We avoided the infinite loop!

How long does this algorithm take? Recall that partitioning is a linear-time operation.

Partitioning [5, 4, 3, 2, 1] took $N = 5$ time.

Then, partitioning [4, 3, 2, 1] took $N - 1$ time.

Then, partitioning [3, 2, 1] took $N - 2$ time.

The array we partition gets smaller by 1 each time, until we reach the size-1 base case subarrays.

Therefore, the total amount of work done is $N + (N - 1) + (N - 2) + \dots + 3 + 2 + 1 = \Theta(N^2)$.

Notice that the right subarrays we create in this case are always size-1, so the amount of work done on the right side is constant-time and does not affect our big-theta bound. (It's kind of similar to the worst-case quicksort, where our subarrays get smaller by 1 each time.)

Best case proof:

On an exam, you might write down $\Theta(N^2)$ as your best-case at this point, since you've tried lots of examples, and $\Theta(N^2)$ was the only non-infinite runtime you could find. If you're running short on time, this can be a good strategy: write down the best-case example you found, and the worst-case example you found, even if you cannot prove that they're really the best and worst cases.

For the sake of completeness, let's argue that you can't do any better than $\Theta(N^2)$.

As we reasoned earlier, if the right subarray ever gets non-trivial (2 or more items), the algorithm infinite-loops. Suppose we have a subarray like [1, 2]. We repeatedly pick 1 as the pivot, and always end up with [1, 2] in the right subarray.

Thus, in order to make the algorithm finish at all, we must avoid the right subarray getting non-trivial.

This means that with a starting array of N items, the right subarray can have at most 1 item, and everything else ($N - 1$ items) must end up in the left subarray.

Then, when sorting the left subarray, we have the same reasoning: The right subarray can have at most 1 item, and everything else ($N - 2$ items) must end up in the left subarray.

This argument helps us ensure that the case we examined (items in reverse order, e.g. [5, 4, 3, 2, 1]) is indeed the only case where the algorithm terminates. Since that case has runtime $\Theta(N^2)$, we are now confident that the best-case runtime is $\Theta(N^2)$, and we are done.

Note: We ignored arrays with duplicates in this entire solution, because the top of the question says we are sorting an array of N distinct items. In general, when analyzing sorting algorithms, it can be

helpful to try the array of all-duplicates, e.g. $[8, 8, 8, 8]$, as it often produces interesting behavior!

- (h) (3 points) What is the runtime behavior if the pivot is always placed at the right-most position in the right subarray?

Best Case:

$$\Theta(N \log N)$$

Worst Case:

$$\Theta(N^2)$$

Worst Case Example:

Just like in the previous subpart, let's begin by sorting some sample inputs with this algorithm.

Let's try $[1, 2, 3, 4, 5]$ again.

The pivot is 1. The left sub-array is empty. The right subarray is $[2, 3, 4, 5, 1]$. Notice that unlike the previous subpart, we now placed the pivot (1) in the right-most position in the right subarray.

Now we need to sort $[2, 3, 4, 5, 1]$. The pivot is 2. The left subarray is $[1]$. The right subarray is $[3, 4, 5, 2]$, with the pivot (2) in the right position.

Now we need to sort $[3, 4, 5, 2]$. The pivot is 3. The left subarray is $[2]$. The right subarray is $[4, 5, 3]$, with the pivot (3) in the right position.

Now we need to sort $[4, 5, 3]$. The pivot is 4. The left subarray is $[3]$. The right subarray is $[5, 4]$, with the pivot (4) in the right position.

Now we need to sort $[5, 4]$. The pivot is 5. The left subarray is $[4]$ and the right subarray is $[5]$.

At this point, all subarrays are of size 1, so we are done.

How long did that sort take? Let's look at what we had to partition:

$[1, 2, 3, 4, 5]$ took N time.

$[2, 3, 4, 5, 1]$ took N time.

$[3, 4, 5, 2]$ took $N - 1$ time.

$[4, 5, 3]$ took $N - 2$ time.

$[5, 4]$ took $N - 3$ time.

It seems like at every step (except the first one), the size of the remaining array decreases by 1. We do create other recursive subproblems along the way, but they're all size-1 trivial problems, so they don't contribute to the total runtime here.

The total work done is $N + N + (N - 1) + (N - 2) + (N - 3) + \dots + 3 + 2 + 1 = \Theta(N^2)$.

Note: If the extra N term concerns you, note that the overall sum is $N + N(N - 1)/2$, and the N is a lower-order term that can be dropped.

Worst Case Analysis:

Can we do any worse than the $\Theta(N^2)$ case we found?

At this point, if this explanation isn't making sense, it's probably a good idea to try out a few more examples before you read further.

After trying more examples, you'll start to spot a pattern. No matter what input you provide, the problem starts breaking up into strictly smaller subproblems after a few recursive steps. This is different from the previous subpart, where you could get stuck with a problem that never gets smaller and infinite-loops.

What's special about this algorithm that causes the problem to eventually break into smaller subproblems?

We are now placing the pivot at the *right-most* position in the right subarray. For example, when the input is $[1, 2, 3, 4, 5]$, the pivot gets placed at the right: $[2, 3, 4, 5, 1]$.

Notice that the pivot is the smallest item in the right subarray (because the right subarray has everything greater than or equal to the pivot).

Since we placed the pivot at the right-most position, the next pivot we choose (left-most position) is definitely not going to be the same. For example, when sorting $[1, 2, 3, 4, 5]$, the pivot was 1, but after partitioning to get $[2, 3, 4, 5, 1]$, the pivot is now 2 (it's guaranteed to not be 1 again).

Also, our new pivot is definitely going to have one item less than it: the old pivot. In the example, when we chose 2 as the new pivot, we know that 1 (old pivot, which is the smallest item in the right subarray) is going to be less than the new pivot. This means that our new subproblem is going to break up into strictly smaller problems, since at least one item is ending up in a left subarray. We won't get the case from earlier where the problem size stays the same forever.

(It's okay if you don't get this exact line of reasoning on an exam. The important thing to notice is that the problem size is always getting strictly smaller after each iteration.)

Since the problem is getting strictly smaller on every iteration, the worst case would be the problem size decreasing by 1 every time, similar to the quicksort worst-case:

A size N problem splits into a size $N - 1$ subproblem and a size 1 subproblem.

The size $N - 1$ subproblem splits into a size $N - 2$ subproblem and a size 1 subproblem.

The size $N - 2$ subproblem splits into a size $N - 3$ subproblem and a size 1 subproblem.

...and so on.

Each partition takes linear time, so the total work done here is $N + (N - 1) + (N - 2) + \dots + 3 + 2 + 1 = \Theta(N^2)$. This convinces us that we have found the worst-case, and we can't do any worse than $\Theta(N^2)$.

Best Case Analysis:

We noticed (through examples or analysis) that the subproblem sizes are getting strictly smaller.

The fastest way to get the problems smaller is just like the quicksort best-case: Try to always split a size N problem into two equally-sized $N/2$ problems.

Let's try to find an example that causes the problems to split into evenly-sized subproblems.

Consider $[4, 2, 1, 3, 6, 5, 7]$.

The pivot is 4. The left subarray is $[2, 1, 3]$. The right subarray is $[6, 5, 7, 4]$, with the pivot (4) on the right.

- Now, we need to sort $[2, 1, 3]$. The pivot is 2. The left subarray is $[1]$. The right subarray is $[3, 2]$, with the pivot (2) on the right.

- Now, we need to sort [3, 2]. The pivot is 3. The left subarray is [2]. The right subarray is [3].
- Now, we need to sort [6, 5, 7, 4]. The pivot is 6. The left subarray is [5, 4]. The right subarray is [7, 6], with the pivot (2) on the right.
 - Now, we need to sort [5, 4]. The pivot is 5. The left subarray is [4]. The right subarray is [5].
 - Now, we need to sort [7, 6]. The pivot is 6. The left subarray is [6]. The right subarray is [7].

How did we come up with this example? We put the median value, 4, on the left, so it will be chosen as the pivot. Then, we know [1, 2, 3] are in the left subarray in some order, so we put [2, 1, 3] to ensure that the median, 2, is on the left and will be chosen as the next pivot. It took some trial-and-error to come up with this example, but we used the guiding principle of “split into equally-sized subproblems” to help us find it. Even if you couldn’t find this example, you can derive the best-case by noticing that it’s possible to create inputs that split problems into equally-sized subproblems.

This example convinces us that we can construct inputs that cause the problems to split into evenly-sized subproblems, just like the quicksort best-case.

The best-case runtime analysis is very similar to the quicksort best-case runtime analysis (exact same recursive tree), and the resulting best-case runtime is $\Theta(N \log N)$.

5 Binary Tries

(16 points)

Consider using a trie to implement a Map, where the keys are restricted to Strings containing only '0's and '1's. Examples:

Valid Keys:	"0101"	"0"	""
Invalid Keys:	"10015"	"den"	"123"

Consider the BinTrieMap class, which implements the trie described above:

```
// Maps strings containing only '0's and '1's to non-null values.
public class BinTrieMap<V> implements Map<String, V> {
    public class Node {
        Node zero; // Child corresponding to 0.
        Node one;  // Child corresponding to 1.
        V value;    // If V is null, the string is not in the map.
                    // If V is not null, the value associated with the string.

        // Creates an empty Node, setting zero, one and value to null.
        public Node() { }
    }

    public Node root;

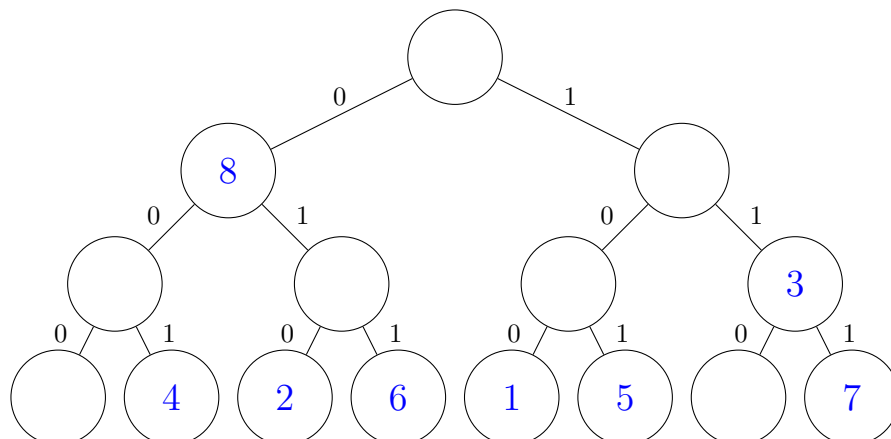
    @Override
    public V put(String key, V value) { /* Part (c) */ }
    ...
}
```

- (a) (6 points) We create a new BinTrieMap<Integer>, and insert the following key-value pairs (from left to right):

Key	"101"	"011"	"100"	"010"	"111"	"001"	"11"	"0"
Value	5	6	1	2	7	4	3	8

In the below diagram of a BinTrieMap, write each inserted value in its corresponding node.

Not all nodes may be used.



For each key-value pair, spell out the key along the edges to find the corresponding node.

For example, if the key is "011", start at the root node. Go to the left child (follow the 0 edge). Then go to the right child (follow the 1 edge). Then go to the right node (follow the 1 edge). This is the node where we put the 6.

For keys that are shorter, just spell out fewer characters (follow fewer edges). For example, if the key is "0", start at the root node, and go to the right child (follow the 0 edge). This is the node where we put the 8.

The way we insert shorter keys is the same way we insert shorter words into a trie (as seen in lecture). When the word is shorter, we finish spelling the word at a “shallower” depth, and that’s fine (words exist at different depths in the trie).

- (b) (2 points) What is the worst-case number of nodes in a `BinTrieMap` with N distinct key-value pairs, where each key is B characters long? Express your answer in Theta notation.

$$\Theta(NB)$$

The worst-case number of nodes occurs when none of our keys share any nodes. For example, suppose we had "0000000...000" and "1111111...111" as keys. Then the resulting trie would have a length- B chain of nodes hanging to the left, and a separate length- B chain of nodes hanging to the right.

Each key has its own length- B chain of nodes. There are N keys in total. Thus, the total number of nodes is $\Theta(NB)$.

Technically, some of the early nodes will end up being shared. 0 and 1 are the only valid characters, so if we have a large number of keys, they’ll all share the 0 and 1 nodes at the first level (directly below the root). However, as both N and B grow large, the deeper nodes are not shared, and we still get the result where every key basically has its own length- B chain of nodes.

If you account for the note above, you’d get $\Theta(\min(NB, 2^B))$ or $\Theta(N(B - \log(N) + 1))$. We also accepted these as fully-correct answers, though they were quite rare.

- (c) (8 points) Fill in the `put` method, which places the given key-value pair into the `BinTrieMap`, and returns the inserted value.

```
public V put(String key, V value) {

    Node n = root;

    for (char c : key.toCharArray()) { // Iterates through every character of key.

        if (c == '0') {
            if (n.zero == null) {

                n.zero = new Node();
            }

            n = n.zero;
        }
    }
}
```

```

        else if (c == '1') {

            if (n.one == null) {

                n.one = new Node();
            }
            n = n.one;
        }
        else {
            throw new IllegalArgumentException("Key must contain only 0s and 1s");
        }
    }
    n.value = value;
    return value; // Returns the inserted value.
}

```

Explanation:

Note that this algorithm is iterative, not recursive.

The for loop helps us spell out each character of the key.

If the character is 0, we move to the left child with `n = n.zero`. Otherwise, we move to the right child with `n = n.one`.

If the desired child doesn't exist (`n.zero == null` in the zero case, `n.one == null` in the one case), then we have to first create that child node with `n.zero = new Node()` in the zero case, or `n.one = new Node()` in the right case.

Alternate Solutions (Blanks 1 and 4):

For Blank 4, swap all the 0s in this section with 1s.

`c == "0"`, i.e. double-quotes instead of single-quotes, does not work. In Java, double-quotes are used for Strings, while single-quotes are used for characters. This line ends up comparing `c`, a character, with `"0"`, a string, which does not work.

`c == 0`, i.e. missing quotes, does not work. This line tries to compare `c`, a character, with `0`, an integer.

`c.equals('0')` does not work, because `c` is of primitive type `char`, and therefore does not have any methods. For example, calling `'q'.equals(...)` does not work, because `'q'` is not an object and has no methods.

Some alternate answers rely on the fact that Java characters are represented as ASCII numbers. Specifically the `'0'` character is represented as number 48, and the `'1'` character is represented as number 49. You were not expected to know this on an exam, and the intended solution doesn't require knowing this.

`c - '0' == 0` is correct. It relies on subtracting the numerical values of characters.

`c == 48` for Blank 1 and `c == 49` for Blank 4 is correct. It relies on the underlying ASCII representations of the characters (and also requires you to memorize the ASCII encodings, which you definitely do not have to do for 61B).

`c == (char) 0`, i.e. casting integer to character, does not work. This line still ends up comparing `c` to the ASCII character represented by number 0 (which is actually something called a null terminator, and not the printable character `'0'`).

Alternate Solutions (Blanks 2, 3, 5, 6):

No alternate solutions that we know of.

6 Natalia MST

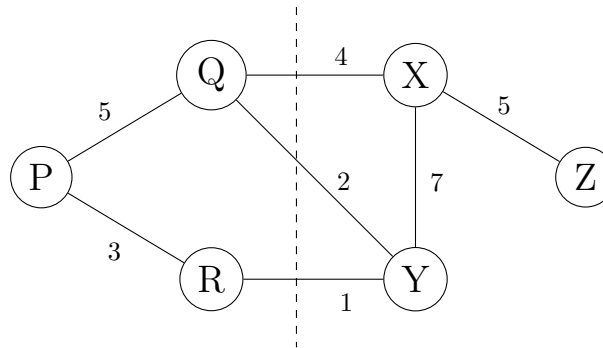
(22 points)

Reminder: Given a graph $G = (V, E)$, a cut $C = (S, T)$ of a graph is two sets of vertices S and T , where every vertex in V is in exactly one of S or T .

Given a connected graph $G = (V, E)$ with **positive** edge weights, and a cut $C = (S, T)$, the **Natalia MST** is defined as follows:

- The Natalia MST is a spanning tree of G .
- **Exactly** one edge of the Natalia MST crosses C .
- The total weight of the tree must be minimal over all spanning trees satisfying the above condition.

(a) (3.5 points) Consider the graph below, with the dotted line representing the cut C .



Select all edges in the resulting Natalia MST of the above graph.

☒ P - Q

☐ Q - X

☒ R - Y

☒ X - Z

☒ P - R

☐ Q - Y

☒ X - Y

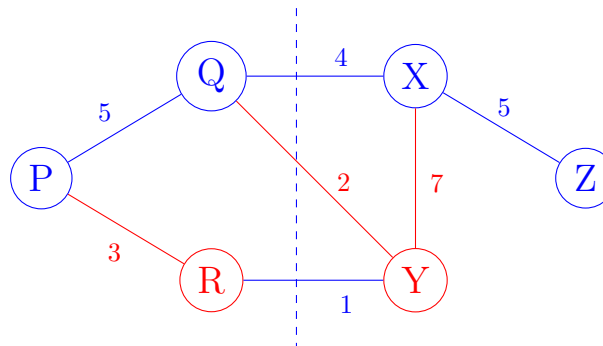
By definition, exactly one edge of the Natalia MST crosses C .

The cut property says that given a cut, we should use the minimum-weight edge crossing that cut in order to minimize the weight of our resulting MST.

The three edges crossing C have weights 4, 2, and 1, so by the cut property, we should use the edge with weight 1. This gives us **R - Y** as one of the edges.

Now that we have a tree that includes R and Y, we can continue building the MST using the conceptual version of Prim's algorithm, while keeping the constraint in mind (namely, we cannot use more edges crossing C).

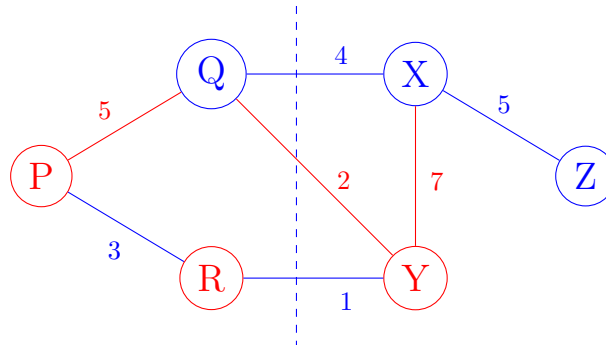
Consider the cut with (R, Y) on one side, and everything else on the other side:



Out of the 3 edges crossing this cut (shown in red above), the smallest edge is $Q - Y$ with weight 2. However, we cannot use this edge, because it crosses C .

We instead use the next-smallest edge crossing the cut, which is $P - R$ with weight 3.

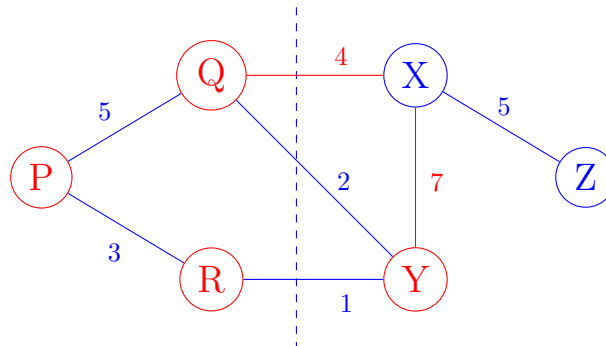
Now we have a cut with (P, R, Y) on one side, and everything else on the other side:



Out of the 3 edges crossing this cut (shown in red above), the smallest edge is $Q - Y$ with weight 2. However, we cannot use this edge, because it crosses C .

We instead use the next-smallest edge crossing the cut, which is $P - Q$ with weight 5.

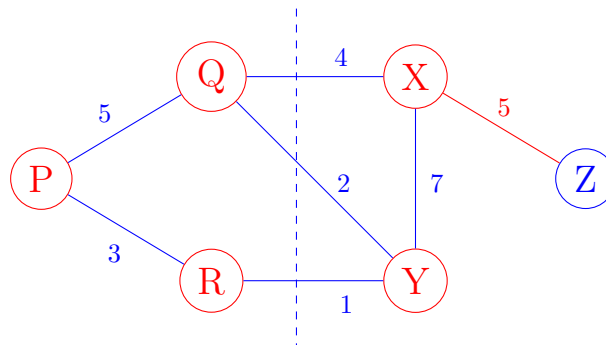
Now we have a cut with (P, R, Y, Q) on one side, and everything else on the other side.



Out of the 3 edges crossing this cut (shown in red above), the smallest edge is $Q - X$ with weight 4. However, we cannot use this edge, because it crosses C .

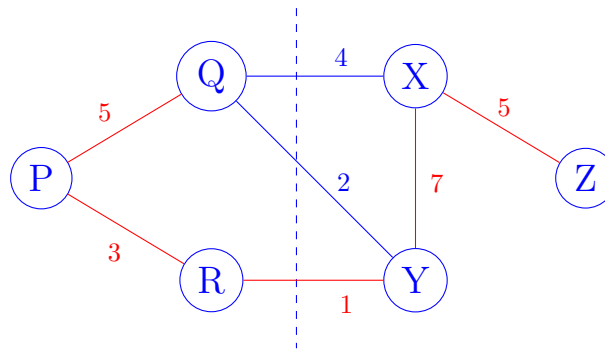
We instead use the next-smallest edge crossing the cut, which is $X - Y$ with weight 7.

Now we have a cut with (P, R, Y, Q, X) on one side, and everything else on the other side.



The only edge crossing this cut is $X - Z$, and this edge does not belong to C , so we can use this cut.

We now have $V - 1 = 5$ edges in our Natalia MST, and all of our edges touch the Natalia MST, so we are done. The completed Natalia MST is shown below:



- (b) (3 points) True/False: For any connected graph G and cut $C = (S, T)$, it is always possible to generate a Natalia MST.

☐ True

☒ False

If you answered True, briefly explain your reasoning in no more than 20 words.

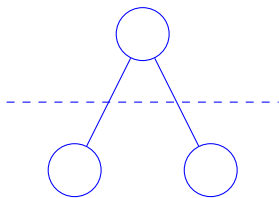
If you answered False, provide a counterexample by describing or drawing a graph with at most 4 nodes (draw the cut C as a dotted line).

Intuitively, building a Natalia MST is impossible if we require two or more edges crossing C in order to touch all of the nodes.

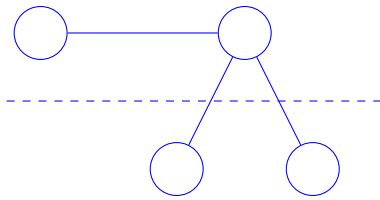
In other words, the Natalia MST is impossible when *any* spanning tree, regardless of weight, requires using two edges crossing C .

Correct counter-examples:

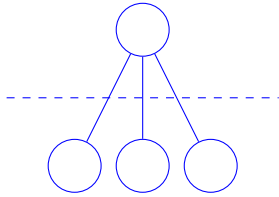
Note that most of these examples are simple enough that you can insert any edge weights, and the counter-example is still valid. For example, many of the graphs require every edge to be in the spanning tree, so it doesn't matter what weights you assign to the edges.



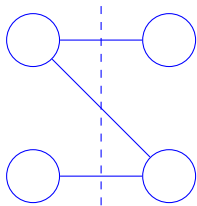
Any spanning tree must use both edges.



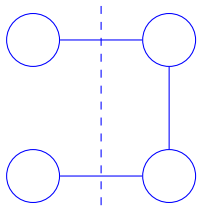
Same example as above, but with an extra fourth node that doesn't change anything.



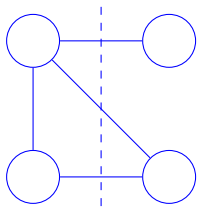
Similar reasoning as above, but with one extra node. Any spanning tree must use all 3 edges.



Any spanning tree must use all 3 edges.



Any spanning tree must use all 3 edges.

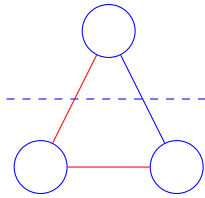


Any spanning tree must use at least 2 of the 3 edges crossing the cut.

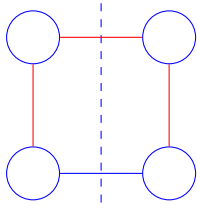
Incorrect counter-examples:

For each of these incorrect answers, a valid Natalia MST is shown in red.

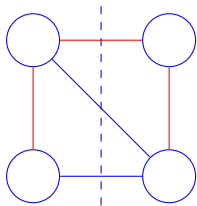
Note that even if you added different edge weights, all of these counter-examples are still incorrect. Recall that the Natalia MST is the minimum out of *all* spanning trees with one edge crossing C . Therefore, as long as there is at least one spanning tree (regardless of weight) with one edge crossing C , then there is a valid Natalia MST.



Only one edge crossing the cut is needed.



Only one edge crossing the cut is needed.



Similar example as above, but with an extra edge added.

Note that valid counter-examples with more than 4 nodes do exist, but we graded them as incorrect, because the question said to use at most 4 nodes.

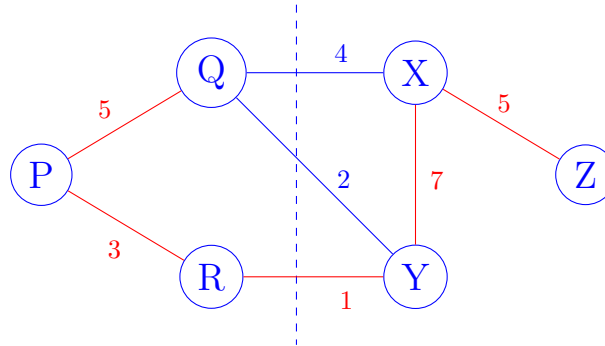
- (c) (1.5 points) True/False: For a given graph G (which has a valid Natalia MST) and a given cut C , its MST (without any restrictions) and its Natalia MST are guaranteed to have the same weight.

☐ True

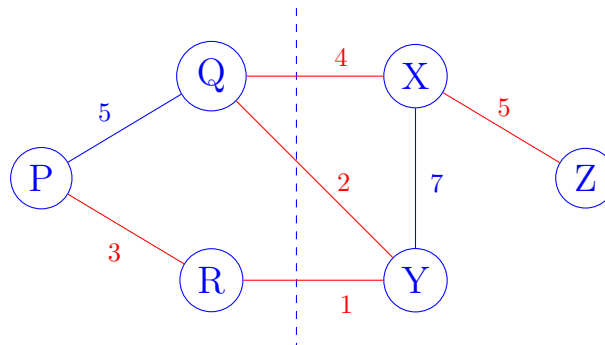
☒ False

False. The simplest counter-example to this statement is the example graph we provided above.

The Natalia MST of the example graph is shown in red below. It has total weight $5 + 3 + 1 + 7 + 5 = 21$.



The MST (without any restrictions) of that same graph is shown in red below. It can be derived by running standard Prim's or Kruskal's. It has total weight $3 + 4 + 2 + 1 + 5 = 15$.



More generally, an MST without any restrictions might use multiple edges crossing C . However, the Natalia MST can only use one edge crossing C , so it may be forced to not use the other low-cost edges crossing C , and use other higher-cost edges instead. This is exactly what happens in the example graph.

- (d) (6 points) Which of the following modified MST algorithms will always produce a valid Natalia MST? Assume it is possible to construct a Natalia MST for the given graph G and cut $C = (S, T)$.

Select all that apply.

☒ Remove all edges across the cut C and run Prim's algorithm separately on the two sets S and T . Then, combine the two resulting MSTs by adding the minimum-weight edge across the cut.

☒ Identify the minimum-weight edge across the cut C , connect its two endpoints, and remove all other edges crossing the cut. Then, run Prim's algorithm starting from one of the connected nodes.

☐ Run Prim's algorithm on the entire graph. After finding the MST, keep only the minimum-weight edge across the cut C , and remove all other edges crossing the cut.

☒ Add k to all edges crossing the cut, where k is the weight of the heaviest edge in the graph. Then, run Kruskal's algorithm.

☐ None of the above

We provide more detailed solutions here for your reference, though you did not have to follow this exact line of reasoning on an exam. When I solved this question, I selected the correct answers because they seemed correct based on intuition, even though I didn't have a full proof.

(A) is true, because it satisfies all the properties of a Natalia MST:

The result is a spanning tree. By definition, the MST of S touches all nodes in S , with no cycles. Similarly, the MST of T touches all nodes in T , with no cycles.

Because we removed all edges across C , the two separate MSTs are disconnected, and adding the minimum-weight edge across C will also not create a cycle.

Therefore, the final result is still a tree (no cycles, all nodes connected), touching every node (the two MSTs combined touch all the nodes).

Exactly one edge crosses the cut. We remove all edges crossing C before building the two separate MSTs, so those two separate MSTs will not have any edges crossing C . Then, we add a single edge crossing C .

The total weight of the tree is minimal. To see why this is true, we consider several cases:

Could we improve by replacing an edge connecting two nodes in S , with another edge connecting two nodes in S ? No, because we already built the MST of the S sub-tree. (Similar argument for replacing an edge connecting two nodes in T , with another edge connecting two nodes in T .)

Could we improve by replacing an edge crossing C with another edge crossing C ? No, because we picked the minimum-weight edge crossing C .

Could we improve by replacing an edge crossing C with an edge connecting nodes in S or T ? Or vice-versa? No, because this would violate the constraint that there is exactly one edge crossing C .

All these cases together demonstrate that there is no way to improve the weight of the tree while preserving the Natalia MST properties. Therefore, the result is a valid Natalia MST with minimal weight.

(B) is true, because it satisfies all the properties of a Natalia MST:

The result is a spanning tree. When you start running Prim's algorithm, you've added two nodes to the tree under construction (whereas standard Prim's starts with one node added). This is equivalent to starting from the second round of Prim's algorithm, where two nodes have been added to the tree under construction. After running Prim's algorithm to completion, the result is still a spanning tree. (In short, it's still Prim's algorithm, just with a slightly more advanced starting state.)

Exactly one edge crosses the cut. We guarantee this by adding only one edge crossing C , and then removing all other edges crossing C before starting Prim's algorithm (so that no further edges crossing C will be added).

The total weight of the tree is minimal. To see why this is true, we again consider several cases:

Could we improve by replacing an edge crossing C with another edge crossing C ? No, because we picked the minimum-weight edge crossing C .

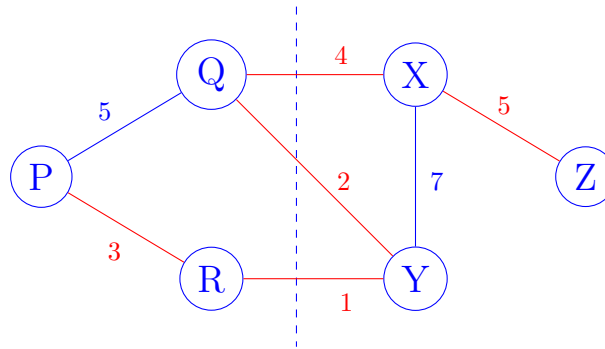
Could we improve by replacing an edge crossing C with an edge not crossing C ? Or vice-versa? No, because this would violate the constraint that there is exactly one edge crossing C .

Could we improve by replacing an edge not crossing C with another edge not crossing C ? No, because every one of these edges was added by Prim's algorithm, and Prim's was using the cut property to add every one of these edges. In other words, every one of the edges is the minimum edge crossing some cut, and replacing the edge would not help us improve.

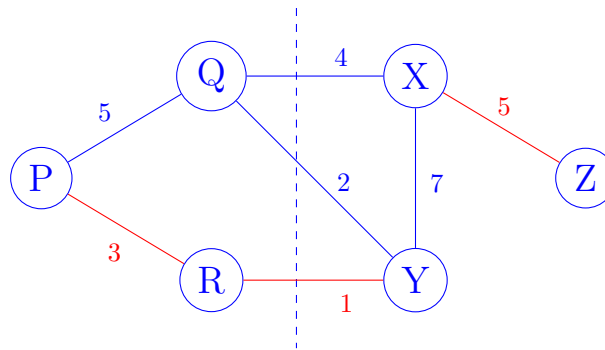
All these cases together demonstrate that there is no way to improve the weight of the tree while preserving the Natalia MST properties. Therefore, the result is a valid Natalia MST with minimal weight.

(C) is false. For a counter-example, we again turn to the example graph. (Thank you exam writers for giving us such a useful graph!)

First, we run Prim's algorithm on the entire graph to get the MST (without any restrictions):



Now, if we only keep the minimum-weight edge crossing C (here, that's the edge with weight 1), and we delete all other edges crossing C , we get this result:



This is not a valid MST because it does not touch all nodes. Equivalently, there are not $V - 1$ edges, which is the number of edges needed in a valid MST.

More generally, this option does not work because Prim's algorithm outputs a MST with exactly $V - 1$ edges, and if you delete any further edges (without adding in any others), the result will have too few edges to connect up all the nodes and form an MST.

(D) is true. Intuitively, this option takes all the edges crossing C , and moves them to the very end of the list of edges considered by Kruskal's algorithm. The list of edges to consider is now everything not crossing C (sorted by increasing weight), followed by everything crossing C (sorted by increasing weight).

Now, consider the behavior of Kruskal's algorithm before it reaches any of the edges crossing C .

Kruskal's considers all of the edges in two separate sub-graphs S and T in increasing order, adding edges as long as no cycles are formed. After considering all non-crossing edges, Kruskal's algorithm has built two separate MSTs for S and T , just like in Option (A)!

In other words, this part of the algorithm is basically running Kruskal's on the two separate sub-graphs, considering the edges in some interleaved order. Sometimes we consider an edge in S and work on that sub-graph (not affecting T at all), and sometimes we consider an edge in T and work on that sub-graph (not affecting S at all). Since the two sub-graphs are separate from each other (adding an edge to one will not affect anything in the other), the result at the end of this part of the algorithm will be valid MSTs on the two sub-graphs.

Finally, we consider the edges crossing C , in increasing weight. Note that only one of these crossing edges will be added by Kruskal's algorithm, because if we added two crossing edges, we'd create a cycle (there would be a way to get from S to T and back to S). So this second part of the algorithm is equivalent to adding the minimum-weight edge across the cut.

In the end, Option (D) produces the same result as Option (A), except it uses Kruskal's instead of Prim's to build the MSTs of the two sub-graphs. Since we already showed in Option (A)'s explanation that its result is a valid Natalia MST, the same holds for Option (D).

- (e) (8 points) You are given a graph G , a cut C , and a corresponding valid Natalia MST, N .

An edge e is deleted from the graph, forming a new graph G' .

Design an algorithm that takes as input G , C , N , e , and G' , and outputs a Natalia MST of the new graph G' .

You may assume that all graphs and Natalia MSTs are represented as adjacency lists. For credit, your algorithm must run in worst-case $O(|E|)$ runtime. You may assume that G' has a valid Natalia MST.

Note: All algorithms from the previous subpart take longer than the required runtime.

This was a very tricky question!

Answer:

There are three cases we must consider.

- **Case I:** The deleted edge e was not in the Natalia MST N . In this case, just output N , the same Natalia MST with no changes.
- **Case II:** The deleted edge e was in N , and was the edge crossing the cut C . In this case, find the next-smallest edge crossing the cut C , and combine that edge with N to form the new Natalia MST.
- **Case III:** The deleted edge e was in N , but was *not* the edge crossing the cut C . In this case, the MST has been split into two disconnected components. Find the smallest edge crossing the new cut (not C) formed by the two disconnected components, and combine that edge with N to form the new Natalia MST.

Explanation:

Coming soon ← I wrote this in December 2024. Turns out “soon” actually meant December 2025. But it’s done now.

With algorithm design questions, it’s difficult to describe one canonical way to reach the solution, so here is a list of assorted thoughts that might go through your head to help get you closer to the answer.

The note tells us that all algorithms from the previous subpart take too long. This makes some intuitive sense, because those algorithms start building the Natalia MST from scratch. However, in this question, we already have a fully-correct Natalia MST (N) as one of our inputs, so it would be silly (and too slow) to throw away N and start over from scratch. We are going to have to use N in some meaningful way to help build our output.

Some algorithms involve “breaking” something and then making a slight adjustment (hopefully with fast runtime) to fix the broken thing. In this case, we notice that deleting e potentially “breaks” the Natalia MST. So a question you might ponder is: If it’s broken, what exactly is broken about it, and how do we fix it?

The answer: We know that MSTs always have $V - 1$ edges. If you delete an edge e , you now have $V - 2$ edges, which is one too few. To fix, you have to add back one more edge to restore the invariant that MSTs have $V - 1$ edges.

So now we’ve boiled down the problem to finding an edge. Sometimes, it can help to think back to previous algorithms we’ve seen for inspiration. So another question you might ponder is: Have we seen any MST-related algorithms where you find an edge to add?

The answer: Prim’s algorithm finds an edge to add by using the cut property! Next question: Do we have a useful cut we can exploit here to find an edge?

Yes—when we delete an edge e out of the Natalia MST, we’ve split the MST into two disconnected parts, which forms a cut! So we can fix the Natalia MST by adding back the smallest edge crossing this cut. This is the key idea in the answer given above.

Runtime check:

First, note that iterating through all edges in an adjacency list takes $\Theta(|E|)$ time, because across all the lists, you see every edge once.

Minor note: Technically you could say this iteration takes $\Theta(|V| + |E|)$ time or $\Theta(\max(|V|, |E|))$ time, but in this question, we know that $|E| \geq |V| - 1$ since there must be at least $|V| - 1$ edges to form a valid Natalia MST in the first place. Therefore, it's safe to say that the iteration takes $\Theta(|E|)$ time, since we will never have so few edges that the $|V|$ term dominates (in that case, the graph wouldn't be connected and no MST would exist).

Also, note that the cut is given as a set (see very top of the question), and finding items in a set can be done in constant time, e.g. with a HashSet.

Now, we consider the three cases and show that each one runs in $O(|E|)$ time.

- **Case I:** We have to check that e is not in the Natalia MST N . Iterating through all the edges in N takes $O(|E|)$ time, since N is a subset of the overall graph and therefore has fewer than $|E|$ edges. (To be really precise, the runtime is $\Theta(|V|) \in O(|E|)$ since there are $|V| - 1$ edges in the Natalia MST.)
- **Case II:** We have to find the smallest edge crossing the cut C . We iterate through each edge in the overall graph, which takes $\Theta(|E|)$ time. For each edge, checking if it crosses the cut is a constant-time operation, since we do two set lookups to check whether the two nodes on either end of the graph are in the same set or different sets. As we check the edges, we can keep a running variable like `best` that remembers the lowest-edge cut crossing C that we have encountered so far.
- **Case III:** Deleting the edge e forms a new cut, not the $C = (S, T)$ cut from earlier. We actually need to do a bit of extra work to figure out the two sets that make up this new cut, but luckily it doesn't add to our runtime.

The deleted edge touches two nodes that are now disconnected. We run DFS from one of the nodes to get a set of all vertices reachable from that node—this forms the first set in the new cut. DFS takes $\Theta(|V| + |E|)$ time, which we can simplify down to $\Theta(|E|)$ time using the reasoning above (i.e. that $|E|$ is roughly greater than $|V|$). We add these nodes into a HashSet, which takes $\Theta(|E|)$ time (we perform $|E|$ constant-time insertions).

Then, we repeat on the other side: We run DFS from the other node touching the deleted edge, and this gives us all vertices reachable from that other node—this forms the other set in the new cut. Again, we add these nodes to a HashSet.

Now that we have a new cut, we have to find the smallest edge crossing the new cut, which follows the same process as in Case II (except that the set lookups are based on the sets of the new cut, not the original cut C). From the reasoning in Case II, we know this step takes $\Theta(|E|)$ time as well.

So in summary, we ran DFS, then ran DFS again, then found the minimum crossing edge. Each of these three steps takes $\Theta(|E|)$ time, and we perform them separately, one after the other, for a total runtime of $\Theta(3|E|) \in \Theta(|E|)$.

Note that you did not have to prove any of this runtime on the exam itself. We are just including the explanation here for completeness and for your enjoyment.

Grading:

Partial credit was awarded for each of the cases.

No credit was awarded for answers that exceeded the runtime requirement. For example, answers that attempted to run Kruskal's algorithm or Prim's algorithm were given no credit (both algorithms take more than $O(|E|)$ runtime).

Note for future semesters: The rubric on this question is not a guarantee of how future questions will be graded.

A rare alternate answer:

Only one student (and none of the TAs) in the Fall 2024 semester came up with this answer. It's not as intuitive as the staff answer, and we don't recommend spending too much time trying to understand it. We are just transcribing their solution here for the sake of completeness. (The transcription itself might have some bugs as well.)

If e is not in N : return N (same as staff answer)

Create a new cut C' with sets R and B .

Label one node that e touches as part of R , and the other node that e touches as part of B .

Define a helper method `travel(curr, prev)`:

- If `prev != null`, set `curr = prev`
- For each node `n` adjacent to `curr`:
 - Add the `n-to-curr` edge to the MST
 - If `n != prev` and the `n-to-curr` edge is in N : recursively call `travel(n, curr)`

Call `travel(e.source, null)`

Call `travel(e.destination, null)` (in other words, call `travel` twice, once for each node touching e)

Find the minimum-weight edge crossing cut C' , treating edges crossing C as having their weight plus the maximum edge weight (as in option (D) from the earlier subpart). This is the edge that we add to N to restore the Natalia MST.

Nothing on this page is worth any points.

Bonus Questions

If you mark A, B, C, and D each with probability $1/2$, what is the probability of marking at least one correct answer to this question? Select all that apply.

☐ (A) $3/4$

☐ (B) $3/4$

☐ (C) $1/4$

☐ (D) $1/8$

Write the name of another class that Aditya could have put in his max-heap. As a reminder, the following class names were already used: Glorp, Foo, Snapp, Jurp, Bram, Wert, Laip.

Foo, Blob, Zorn, Romf, Pookiewookiebear, Fatt, Clob, Gauri, WhoopDeeDoo, Poo, JasonFunderberker, Shnorckle, Shroomp, Aditya, Votty, Humberto, Denero, Bako, Boo, Burgers, Musmus, Srump, Gerald, Jinx, Meep, Gaga, Bloop, God, Poop, Bar, Morp, Croop, Dave, Congor, Oski, LeBron, Flomp, NoClue, CS61B, Stitch, Worcestershire, Zimzem, Group, MaryPoppins, Bob, Gleek, JMoneyYok, Dwum, Sebastian, Yike, Horp, Pymb, Splurt, Juan, Swang, Beep, Gleep, Mea, No, Bruh, Crag, Cain, Igum, Computer, Murry, Zawg, Planet, Ce, Object, Floop, Teff, Kobe, Corgi, Meep, Ditto, Gleebus, Ghyxsqckuz, Blurp, Kora, Foosnapper, Wug, Plop, MoheyLeeMiller, Merp, Harry, Blorp, Yokeyrin, CrackleAndPop, Merf, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Feedback

(0 Points)

Leave any feedback, comments, concerns, or more drawings below!

For those of you studying this exam in future semesters: Good luck on your final exam!